TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
**TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG**
**KHOA CÔNG NGHỆ THÔNG TIN**




**NGÔ TẤN THÀNH - 524H0031**
**TRẦN GIA HUY - 524H0092**
**NGUYỄN QUANG TỰU - 524H0136**

# PHÂN HỆ ĐÓNG HỌC PHÍ CỦA ỨNG DỤNG iBANKING

**BÁO CÁO GIỮA KỲ**
**KIẾN TRÚC HƯỚNG DỊCH VỤ**

**THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026**

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
**TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG**
**KHOA CÔNG NGHỆ THÔNG TIN**




**NGÔ TẤN THÀNH - 524H0031**
**TRẦN GIA HUY - 524H0092**
**NGUYỄN QUANG TỰU - 524H0136**

# PHÂN HỆ ĐÓNG HỌC PHÍ CỦA ỨNG DỤNG iBANKING

**BÁO CÁO GIỮA KỲ**
**KIẾN TRÚC HƯỚNG DỊCH VỤ**

Người hướng dẫn
**TS. Dương Hữu Phúc**

**THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026**

# LỜI CẢM ƠN

Chúng em xin chân thành cảm ơn Ban Giám hiệu và quý Thầy, Cô Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng đã tạo điều kiện học tập và nghiên cứu thuận lợi cho chúng em trong suốt quá trình học tập môn Kiến trúc hướng dịch vụ. Đặc biệt, chúng em xin bày tỏ lòng biết ơn sâu sắc đến TS. Dương Hữu Phúc, người đã trực tiếp giảng dạy, định hướng chuyên môn và hướng dẫn chúng em hoàn thành đề tài báo cáo giữa kỳ "Phân hệ đóng học phí của ứng dụng iBanking". Mặc dù nhóm đã cố gắng hoàn thiện hệ thống và tài liệu báo cáo theo đúng yêu cầu kỹ thuật, bài làm khó tránh khỏi những hạn chế nhất định. Chúng em rất mong nhận được những ý kiến đóng góp của Thầy để đề tài được hoàn thiện hơn.

*TP. Hồ Chí Minh, ngày 08 tháng 11 năm 2026*

*Tác giả*

*Ký tên và ghi rõ họ tên*

***THÀNH***

Ngô Tấn Thành

***HUY***

Trần Gia Huy

***TỰU***

Nguyễn Quang Tựu

## CÔNG TRÌNH ĐƯỢC HOÀN THÀNH

## TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng tôi xin cam đoan đây là công trình nghiên cứu và phát triển phần mềm của nhóm chúng tôi dưới sự hướng dẫn khoa học của TS. Dương Hữu Phúc. Các nội dung thiết kế, mã nguồn hiện thực và kết quả kiểm thử trong đề tài này là trung thực. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính nhóm tác giả thực hiện và thu thập từ các nguồn tài liệu có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong dự án còn tham khảo một số kiến thức kiến trúc phần mềm và tài liệu kỹ thuật từ các nguồn chính thống đều có trích dẫn đầy đủ.

Nếu phát hiện có bất kỳ sự gian lận nào, chúng tôi xin chịu trách nhiệm về nội dung dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do chúng tôi gây ra trong quá trình thực hiện nếu phát sinh vi phạm.

*TP. Hồ Chí Minh, ngày 08 tháng 11 năm 2026*

*Tác giả*

*Ký tên và ghi rõ họ tên*

***THÀNH***

Ngô Tấn Thành

***HUY***

Trần Gia Huy

***TỰU***

Nguyễn Quang Tựu

# MỤC LỤC

**DANH MỤC HÌNH VẼ ..... 4**

**DANH MỤC BẢNG BIỂU ..... 5**

**CHƯƠNG 1. PHÂN TÍCH NGHIỆP VỤ VÀ MÔ HÌNH HÓA ..... 6**

1.1 Danh sách tác nhân ..... 6

1.2 Danh sách Use Case ..... 6

1.3 Use Case Diagram ..... 8

1.4 Danh sách thực thể dữ liệu ..... 9

1.5 Mối quan hệ và ràng buộc bản số ..... 11

1.6 Entity Relationship Diagram ..... 13

**CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC MICROSERVICES ..... 15**

2.1 Phân rã dịch vụ và trách nhiệm của từng dịch vụ ..... 15

2.2 Giao tiếp giữa các dịch vụ, cơ sở dữ liệu và dịch vụ ngoài ..... 16

2.3 Giải thích lựa chọn kiến trúc ..... 18

2.4 Microservices Architecture Diagram ..... 19

**CHƯƠNG 3. THIẾT KẾ REST API ..... 21**

3.1 Quy chuẩn thiết kế REST API chung của hệ thống ..... 21

3.2 Thiết kế REST API của Account Service ..... 23

3.3 Thiết kế REST API của Tuition Service ..... 27

3.4 Thiết kế REST API của Payment Service ..... 29

3.5 Thiết kế REST API của Notification Service ..... 32

3.6 Cơ chế kiểm soát luồng và bảo vệ API tại API Gateway ..... 34

**CHƯƠNG 4. HIỆN THỰC CƠ SỞ DỮ LIỆU ..... 39**

4.1 Nguyên tắc hiện thực cơ sở dữ liệu và lưu trữ phân tán .....39
4.2 Hiện thực cơ sở dữ liệu của Account Service .....41
4.3 Hiện thực cơ sở dữ liệu của Tuition Service.....42
4.4 Hiện thực cơ sở dữ liệu của Payment Service.....42
4.5 Hiện thực cơ sở dữ liệu của Notification Service .....43
4.6 Dữ liệu mẫu khởi tạo và cơ chế lưu trữ bền vững.....43
**CHƯƠNG 5. HIỆN THỰC QUY TRÌNH THANH TOÁN .....46**
5.1 Chức năng đăng nhập, thông tin tài khoản và tra cứu học phí.....46
5.2 Khởi tạo giao dịch, kiểm tra số dư và trạng thái học phí .....47
5.3 Quy trình sinh mã OTP, kiểm soát thời hạn và gửi email .....48
5.4 Hoàn tất thanh toán, gạch nợ và gửi biên nhận xác nhận.....49
**CHƯƠNG 6. XỬ LÝ GIAO DỊCH VÀ ĐỒNG THỜI .....52**
6.1 Đảm bảo tính nhất quán dữ liệu giữa các dịch vụ .....52
6.2 Xử lý nhiều giao dịch đồng thời trên cùng một tài khoản.....54
6.3 Xử lý nhiều tài khoản cùng thanh toán một khoản học phí.....55
6.4 Cơ chế chống gửi trùng yêu cầu bằng Idempotency-Key tại API Gateway.....56
**CHƯƠNG 7. HIỆN THỰC GIAO DIỆN NGƯỜI DÙNG.....58**
7.1 Cấu trúc ứng dụng Web và cơ chế tích hợp với Backend API .....58
7.2 Giao diện đăng nhập.....60
7.3 Giao diện tra cứu học phí và khởi tạo thanh toán.....60
7.4 Giao diện xác thực OTP và nhận email trên Mailpit .....62
7.5 Giao diện kết quả biên nhận và lịch sử giao dịch.....64
**CHƯƠNG 8. TRIỂN KHAI VÀ KIỂM THỬ HỆ THỐNG .....66**

8.1 Hướng dẫn cài đặt, cấu hình môi trường và khởi chạy ..... 66

8.2 Đóng gói và tích hợp hệ thống bằng Docker Compose ..... 68

8.3 Kiểm thử tự động các kịch bản giao dịch đồng thời ..... 70

*8.3.1 Kịch bản 0: Bảo vệ luồng nghiệp vụ Workflow Guard ..... 70*

*8.3.2 Kịch bản 1: Giao dịch đồng thời trên cùng tài khoản ..... 71*

*8.3.3 Kịch bản 2: Đồng thời đóng học phí và bù trừ Saga ..... 72*

*8.3.4 Kịch bản 3: Chống gửi trùng yêu cầu Idempotency-Key ..... 73*

**TÀI LIỆU THAM KHẢO ..... 74**

# DANH MỤC HÌNH VẼ

Hình 1.1: Use Case Diagram của phân hệ đóng học phí iBanking ..... 5

Hình 1.2: Entity Relationship Diagram của phân hệ đóng học phí iBanking ..... 9

Hình 2.1: Sơ đồ kiến trúc Microservices của phân hệ đóng học phí iBanking ..... 10

Hình 4.1: Sơ đồ kiến trúc cơ sở dữ liệu phân tán của các Microservices ..... 28

Hình 5.1: Sơ đồ quy trình thanh toán học phí hai pha và xác thực OTP ..... 33

Hình 6.1: Sơ đồ máy trạng thái giao dịch và cơ chế bù trừ Saga ..... 38

Hình 6.2: Biểu đồ tuần tự luồng thanh toán học phí thành công ..... 40

Hình 6.3: Biểu đồ tuần tự xử lý xung đột đồng thời và bù trừ hoàn tiền Saga ..... 44

Hình 7.1: Giao diện đăng nhập hệ thống iBanking ..... 47

Hình 7.2: Giao diện tra cứu công nợ học phí và thông tin thanh toán ..... 48

Hình 7.3: Giao diện hộp thoại xác thực OTP với đồng hồ đếm ngược ..... 49

Hình 7.4: Giao diện máy chủ kiểm thử Mailpit nhận thư mã OTP và biên nhận ..... 50

Hình 7.5: Giao diện lịch sử giao dịch và biên nhận thanh toán học phí ..... 51

Hình 8.1: Kết quả thực thi Kịch bản 0 - Chặn truy cập trực tiếp API nội bộ tại API Gateway ..... 55

Hình 8.2: Kết quả thực thi Kịch bản 1 - Giao dịch đồng thời trên cùng một tài khoản ..... 56

Hình 8.3: Kết quả thực thi Kịch bản 2 - Hai tài khoản cùng đóng một khoản học phí và bù trừ Saga ..... 57

Hình 8.4: Kết quả thực thi Kịch bản 3 - Khóa bất biến Idempotency-Key tại API Gateway ..... 58

# DANH MỤC BẢNG BIỂU

Bảng 1.1: Danh sách tác nhân của phân hệ ..... 3

Bảng 1.2: Danh sách Use Case của phân hệ ..... 3

Bảng 1.3: Danh sách thực thể dữ liệu của phân hệ ..... 5

Bảng 1.4: Mối quan hệ và ràng buộc bản số giữa các thực thể ..... 7

Bảng 2.1: Danh mục dịch vụ và phân công trách nhiệm ..... 9

Bảng 2.2: Giao tiếp giữa các dịch vụ, cơ sở dữ liệu và dịch vụ ngoài ..... 11

Bảng 3.1: Quy định mã trạng thái HTTP chuẩn của hệ thống ..... 16

Bảng 3.2: Danh mục REST API của Account Service ..... 17

Bảng 3.3: Danh mục REST API của Tuition Service ..... 19

Bảng 3.4: Danh mục REST API của Payment Service ..... 21

Bảng 3.5: Danh mục REST API của Notification Service ..... 23

Bảng 3.6: Quy tắc định tuyến và bộ lọc bảo vệ tại API Gateway ..... 24

Bảng 4.1: Danh mục cơ sở dữ liệu và thực thể phân tán ..... 26

Bảng 4.2: Dữ liệu mẫu tài khoản người dùng trong bảng accounts ..... 31

Bảng 4.3: Dữ liệu mẫu công nợ học phí sinh viên trong bảng tuitions ..... 31

Bảng 5.1: Ma trận kiểm soát điều kiện và xử lý ngoại lệ trong quy trình thanh toán 36

Bảng 7.1: Ma trận ánh xạ tương tác giao diện người dùng và API Microservices 46

Bảng 8.1: Cấu hình triển khai 8 containers trong Docker Compose ..... 51

# CHƯƠNG 1. PHÂN TÍCH NGHIỆP VỤ VÀ MÔ HÌNH HÓA

## 1.1 Danh sách tác nhân

Tác nhân là vai trò của người dùng hoặc hệ thống bên ngoài tương tác trực tiếp với phân hệ qua đường biên hệ thống. Bảng 1.1 trình bày danh sách các tác nhân của phân hệ.

Bảng 1.1: Danh sách tác nhân của phân hệ

| Tác nhân | Mô tả |
|---|---|
| Người dùng | Người sử dụng ứng dụng iBanking thực hiện đăng nhập, xem thông tin tài khoản, tra cứu học phí theo mã số sinh viên, xác nhận thanh toán học phí, nhập mã OTP và xem lịch sử giao dịch. |
| Hệ thống quản lý học phí | Hệ thống quản lý thông tin sinh viên và công nợ học phí của trường TDTU, cung cấp dữ liệu học phí theo mã số sinh viên khi tra cứu và cập nhật trạng thái học phí thành đã thanh toán. |
| Hệ thống Email | Hệ thống thư điện tử tiếp nhận yêu cầu từ phân hệ để gửi email mã OTP xác thực giao dịch và email xác nhận giao dịch thành công đến địa chỉ email của người nộp tiền. |

## 1.2 Danh sách Use Case

Bảng 1.2 liệt kê danh sách các Use Case của phân hệ đóng học phí iBanking cùng tác nhân tham gia và mô tả vắn tắt.

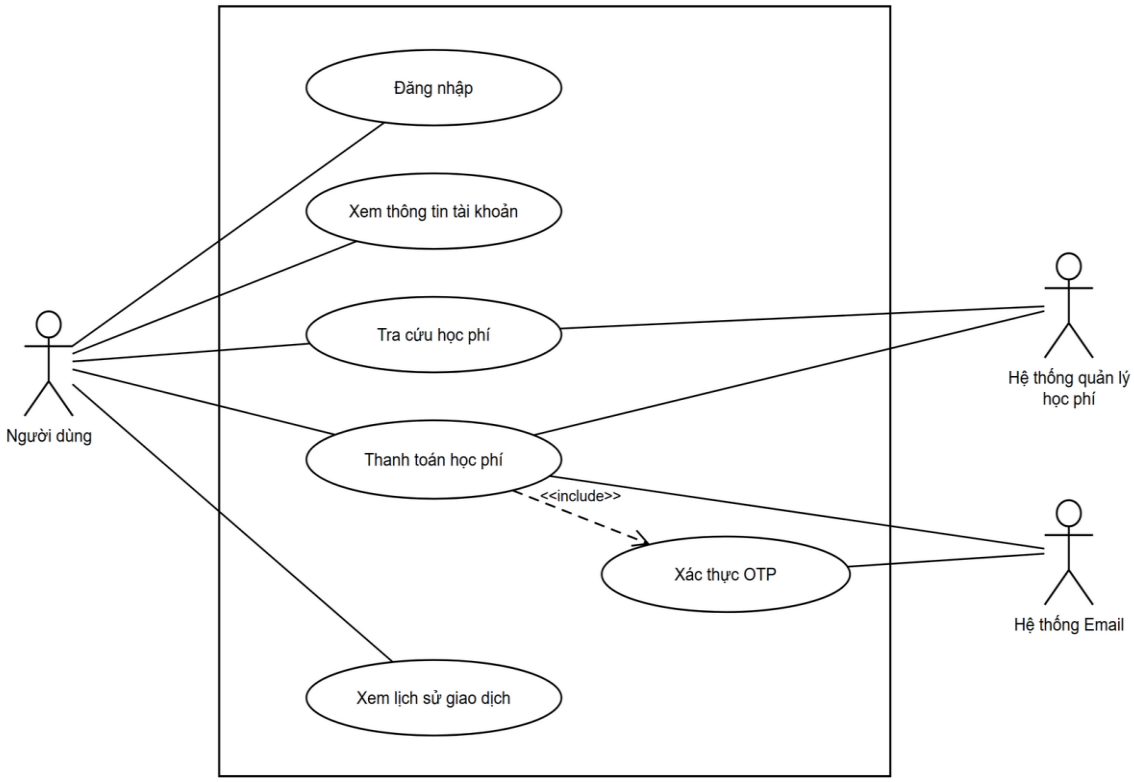
Bảng 1.2: Danh sách Use Case của phân hệ

| Mã UC | Tên Use Case | Tác nhân | Mô tả vắn tắt |
|---|---|---|---|
| UC01 | Đăng nhập | Người dùng | Người dùng đăng nhập vào hệ thống bằng username và password để truy cập chức năng Thanh toán học phí. |
| UC02 | Xem thông tin tài khoản | Người dùng | Hệ thống hiển thị thông tin người dùng gồm họ và tên, số điện thoại, địa chỉ email và số dư khả dụng của tài khoản đang đăng nhập. |
| UC03 | Tra cứu học phí | Người dùng, Hệ thống quản lý học phí | Người dùng nhập mã số sinh viên; hệ thống tự động tra cứu từ Hệ thống quản lý học phí và hiển thị họ và tên sinh viên cùng số tiền học phí còn phải thanh toán. |
| UC04 | Thanh toán học phí | Người dùng, Hệ thống quản lý học phí, Hệ thống Email | Người dùng xác nhận giao dịch thanh toán toàn bộ khoản học phí; hệ thống kiểm tra điều kiện hợp lệ, thực hiện bước Xác thực OTP, trừ số dư tài khoản, cập nhật khoản học phí thành đã thanh toán, lưu lịch sử giao dịch và gửi email xác nhận giao dịch thành công. |

| Mã UC | Tên Use Case | Tác nhân | Mô tả vắn tắt |
|---|---|---|---|
| UC05 | Xác thực OTP | Người dùng, Hệ thống Email | Hệ thống sinh mã OTP gắn với giao dịch gửi qua Hệ thống Email đến địa chỉ email của người nộp tiền và kiểm tra tính hợp lệ, thời hạn hiệu lực tối đa 5 phút của mã OTP do người dùng nhập. |
| UC06 | Xem lịch sử giao dịch | Người dùng | Hệ thống hiển thị danh sách các giao dịch đã thực hiện của người dùng. |

## 1.3 Use Case Diagram

Hình 1.1 biểu diễn Use Case Diagram của phân hệ đóng học phí iBanking. Phân hệ có 3 tác nhân: Người dùng, Hệ thống quản lý học phí và Hệ thống Email. Quan hệ «include» từ Use Case Thanh toán học phí đến Use Case Xác thực OTP chỉ định bước xác thực OTP là điều kiện bắt buộc của mọi giao dịch thanh toán.




*Hình 1.1: Use Case Diagram của phân hệ đóng học phí iBanking*

## 1.4 Danh sách thực thể dữ liệu

Dữ liệu của phân hệ gồm 6 thực thể được phân bổ trên 4 cơ sở dữ liệu tương ứng với các dịch vụ: account_db, tuition_db, payment_db và notification_db. Bảng 1.3 mô tả các thực thể, khóa và thuộc tính dữ liệu.

Bảng 1.3: Danh sách thực thể dữ liệu của phân hệ

| **Thực thể** | **Cơ sở dữ liệu** | **Khóa chính** | **Khóa duy nhất / Khóa ngoại** | **Thuộc tính và mô tả** |
|---|---|---|---|---|
| accounts | account_db | id | UK: username | Thông tin người dùng và số dư: username, password_hash, full_name, email, phone, balance, created_at, updated_at. |
| account_transactions | account_db | id | FK: account_id; UK: account_id, reference_id, transaction_type | Lịch sử biến động số dư tài khoản: transaction_type, amount, balance_before, balance_after, reference_id, description, created_at. |
| students | tuition_db | student_id | - | Thông tin sinh viên: student_id, student_name, major, created_at. |

| Thực thể | Cơ sở dữ liệu | Khóa chính | Khóa duy nhất / Khóa ngoại | Thuộc tính và mô tả |
|---|---|---|---|---|
| tuitions | tuition_db | id | FK: student_id; UK: student_id, semester | Khoản học phí của sinh viên: student_id, semester, amount, status, paid_at, created_at. |
| payments | payment_db | id | FK: account_id, tuition_id | Giao dịch thanh toán học phí: account_id, tuition_id, amount, status, error_code, created_at, completed_at. |
| otps | notification_db | id | FK: payment_id | Mã xác thực OTP: payment_id, otp_code, expires_at, is_used, attempts, created_at. |

## 1.5 Mối quan hệ và ràng buộc bản số

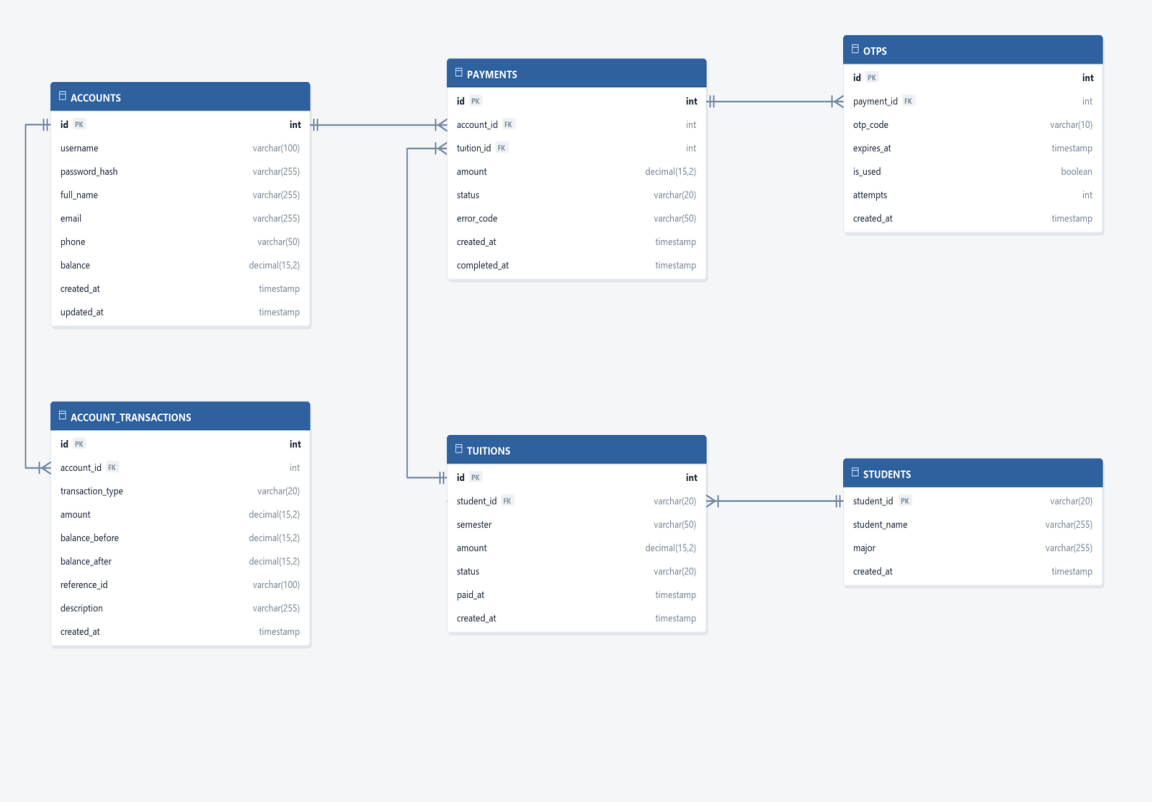
Bảng 1.4 trình bày các mối quan hệ giữa các thực thể dữ liệu và ràng buộc bản số tối thiểu, tối đa.

Bảng 1.4: Mối quan hệ và ràng buộc bản số giữa các thực thể

| Cặp thực thể | Thuộc tính liên kết | Bản số | Mô tả quy tắc dữ liệu |
|---|---|---|---|
| accounts - account_transactions | account_id | 1..1 - 0..* | Một tài khoản có không hoặc nhiều bản ghi biến động số dư; mỗi bản ghi biến động số dư thuộc về một tài khoản. |
| students - tuitions | student_id | 1..1 - 1..* | Một sinh viên có một hoặc nhiều khoản học phí theo học kỳ; mỗi khoản học phí thuộc về một sinh viên. |
| accounts - payments | account_id | 1..1 - 0..* | Một tài khoản có thể thực hiện không hoặc nhiều giao dịch thanh toán học phí; mỗi giao dịch thanh toán do một tài khoản thực hiện. |
| tuitions — payments | tuition_id | 1..1 - 0..* | Một khoản học phí có không hoặc nhiều giao dịch thanh toán liên quan, trong đó tối đa một giao dịch thành công; mỗi giao dịch thanh toán gắn với một khoản học phí. |
| payments - otps | payment_id | 1..1 - 1..* | Mỗi giao dịch thanh toán khi xác nhận có ít nhất một mã OTP được sinh ra; mỗi mã OTP được gắn với một giao dịch cụ thể. |
| payments - account_transactions | reference_id | 1..1 - 0..* | Một giao dịch thanh toán có không hoặc nhiều bản ghi biến động số dư tương ứng với thao tác trừ tiền và hoàn tiền; mỗi bản ghi biến động số dư tham chiếu đến một giao dịch. |

## 1.6 Entity Relationship Diagram

Hình 1.2 trình bày Entity Relationship Diagram của phân hệ đóng học phí iBanking theo ký hiệu Crow's Foot.




*Hình 1.2: Entity Relationship Diagram của phân hệ đóng học phí iBanking*

Phân tích các ràng buộc nghiệp vụ cốt lõi từ mô hình Use Case và ERD: Hệ thống xác lập ranh giới quyền hạn giữa tác nhân Người dùng (phụ huynh/sinh viên nộp tiền) và Giảng viên/Nhà trường (quản trị công nợ). Trong đó, Use Case Đóng học phí mang tính chất của một giao dịch tài chính phân tán quan trọng nhất, đòi hỏi sự phối hợp đồng bộ giữa 4 thực thể: tài khoản người nộp (accounts), bản ghi công nợ học phí (tuitions), phiên giao dịch trung gian (payments) và mã xác thực 2FA (otps).

Nguyên tắc toàn vẹn dữ liệu trong mô hình quan hệ phân tán: Điểm then chốt trong thiết kế ERD là việc phân rã khóa ngoại cứng giữa các bảng vật lý để phù hợp với kiến trúc Database-per-Service. Thay vào đó, tính toàn vẹn liên dịch vụ được duy trì thông qua các thuộc tính tham chiếu logic (account_id, student_id, tuition_id) và đặc biệt là chỉ mục trạng thái `status IN ('UNPAID', 'PAID')` trên bảng tuitions. Ràng buộc logic này chính là cơ sở dữ liệu học thuật vững chắc cho việc xử lý xung đột đồng thời và kích hoạt quy trình bù trừ hoàn tiền Saga ở các chương tiếp theo.

# CHƯƠNG 2. THIẾT KẾ KIẾN TRÚC MICROSERVICES

## 2.1 Phân rã dịch vụ và trách nhiệm của từng dịch vụ

Dựa trên danh sách Use Case và các thực thể dữ liệu đã xác định ở Chương 1, phân hệ đóng học phí iBanking được phân rã thành cổng giao tiếp API Gateway và 4 dịch vụ gồm Account Service, Tuition Service, Payment Service và Notification Service. Bảng 2.1 trình bày danh sách các dịch vụ, cơ sở dữ liệu tương ứng và trách nhiệm của từng dịch vụ.

Bảng 2.1: Danh mục dịch vụ và phân công trách nhiệm

| Tên dịch vụ | Cơ sở dữ liệu | Trách nhiệm |
|---|---|---|
| API Gateway | Không sử dụng | Tiếp nhận yêu cầu từ Web Client, kiểm tra thông tin xác thực và chuyển tiếp yêu cầu đến Account Service, Tuition Service và Payment Service. |
| Account Service | account_db | Quản lý thực thể accounts và account_transactions; xử lý đăng nhập, cung cấp thông tin người nộp tiền, kiểm tra số dư khả dụng, trừ số dư tài khoản và hoàn tiền khi giao dịch không thành công. |
| Tuition Service | tuition_db | Quản lý thực thể students và tuitions; tra cứu thông tin sinh viên và khoản học phí theo mã số sinh viên, kiểm tra trạng thái học phí và cập nhật khoản học phí thành đã thanh toán. |
| Payment Service | payment_db | Quản lý thực thể payments; khởi tạo giao dịch thanh toán, điều phối quá trình xác thực OTP, trừ tiền tài khoản, gạch nợ học phí, lưu kết quả giao dịch và cung cấp lịch sử giao dịch của người dùng. |

| Tên dịch vụ | Cơ sở dữ liệu | Trách nhiệm |
|---|---|---|
| Notification Service | notification_db | Quản lý thực thể otps; sinh mã OTP gắn với giao dịch có thời hạn 5 phút và sử dụng một lần, kiểm tra mã OTP do người dùng nhập và gửi email mã OTP cùng email xác nhận giao dịch thành công. |

## 2.2 Giao tiếp giữa các dịch vụ, cơ sở dữ liệu và dịch vụ ngoài

Các thành phần trong phân hệ trao đổi dữ liệu qua 3 nhóm kết nối:

- **Giao tiếp giữa các dịch vụ:** Web Client gửi yêu cầu đến API Gateway qua giao thức HTTP/REST. API Gateway chuyển tiếp yêu cầu đến Account Service, Tuition Service và Payment Service. Trong quá trình xử lý thanh toán, Payment Service gửi yêu cầu HTTP/REST đến Tuition Service, Account Service và Notification Service để thực hiện các bước kiểm tra học phí, sinh và xác thực mã OTP, trừ số dư, cập nhật trạng thái học phí và gửi email thông báo.
- **Giao tiếp với cơ sở dữ liệu:** Mỗi dịch vụ kết nối và truy vấn cơ sở dữ liệu thuộc phạm vi quản lý của dịch vụ đó qua ngôn ngữ truy vấn SQL, cụ thể: Account Service kết nối account_db, Tuition Service kết nối tuition_db, Payment Service kết nối payment_db và Notification Service kết nối notification_db.
- **Giao tiếp với dịch vụ ngoài:** Notification Service kết nối đến máy chủ SMTP Server qua giao thức SMTP để gửi email chứa mã OTP và email xác nhận giao dịch thành công đến địa chỉ email của người nộp tiền.

Bảng 2.2: Giao tiếp giữa các dịch vụ, cơ sở dữ liệu và dịch vụ ngoài

| **Thành phần nguồn** | **Thành phần đích** | **Giao thức** | **Nội dung giao tiếp** |
|---|---|---|---|
| Web Client | API Gateway | HTTP/REST | Gửi yêu cầu đăng nhập, xem thông tin tài khoản, tra cứu học phí, khởi tạo thanh toán, xác thực OTP và xem lịch sử giao dịch. |
| API Gateway | Account Service | HTTP/REST | Chuyển tiếp yêu cầu đăng nhập, lấy thông tin người nộp tiền và lịch sử biến động số dư. |
| API Gateway | Tuition Service | HTTP/REST | Chuyển tiếp yêu cầu tra cứu thông tin sinh viên và khoản học phí theo mã số sinh viên. |
| API Gateway | Payment Service | HTTP/REST | Chuyển tiếp yêu cầu khởi tạo giao dịch, xác nhận mã OTP và truy vấn lịch sử giao dịch. |
| Payment Service | Tuition Service | HTTP/REST | Kiểm tra khoản học phí khi khởi tạo giao dịch và cập nhật trạng thái học phí thành đã thanh toán. |
| Payment Service | Account Service | HTTP/REST | Kiểm tra số dư khả dụng, trừ số tiền tương ứng khỏi tài khoản người nộp tiền và hoàn tiền khi gạch nợ học phí không thành công. |

| Thành phần nguồn | Thành phần đích | Giao thức | Nội dung giao tiếp |
|---|---|---|---|
| Payment Service | Notification Service | HTTP/REST | Yêu cầu sinh mã OTP gửi qua email, kiểm tra tính hợp lệ của mã OTP và yêu cầu gửi email xác nhận giao dịch thành công. |
| Account Service | account_db | SQL | Đọc và cập nhật dữ liệu bảng accounts và account_transactions. |
| Tuition Service | tuition_db | SQL | Đọc và cập nhật dữ liệu bảng students và tuitions. |
| Payment Service | payment_db | SQL | Đọc và cập nhật dữ liệu bảng payments. |
| Notification Service | notification_db | SQL | Đọc và cập nhật dữ liệu bảng otps. |
| Notification Service | SMTP Server | SMTP | Gửi email chứa mã xác thực OTP và email xác nhận giao dịch thành công đến người nộp tiền. |

## 2.3 Giải thích lựa chọn kiến trúc

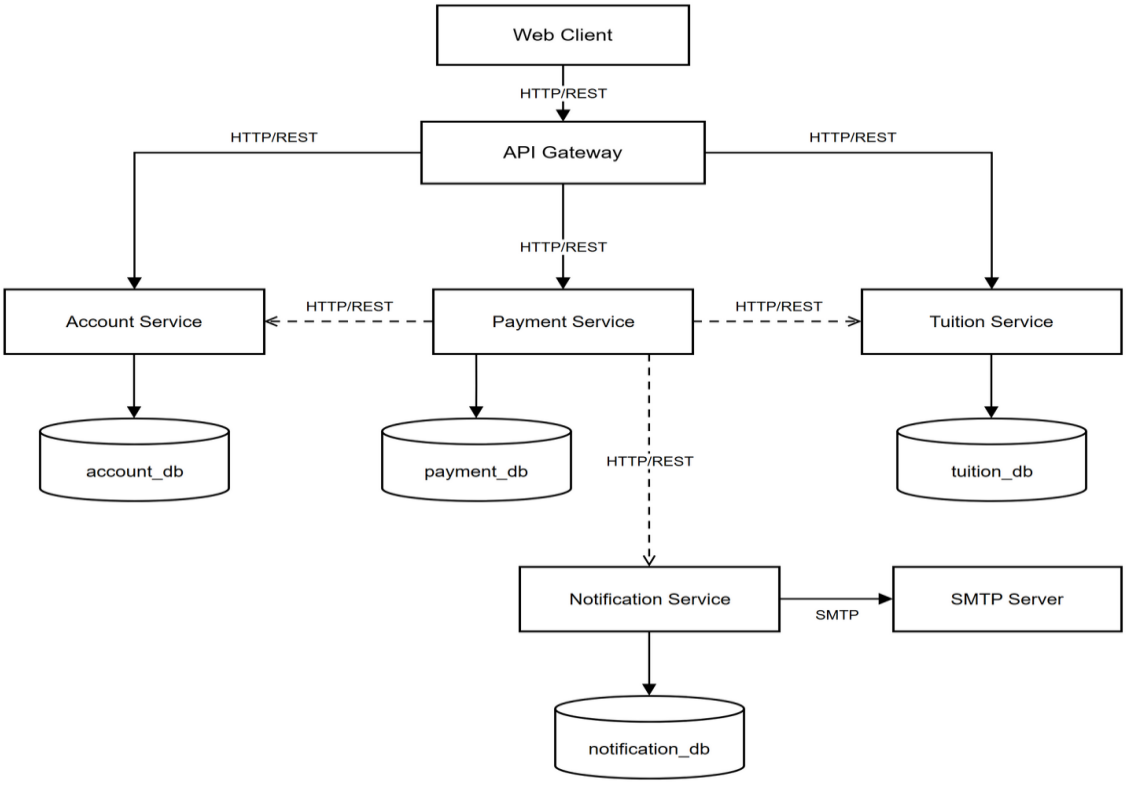
Thiết kế kiến trúc của phân hệ được xây dựng dựa trên 3 lựa chọn:

- **Phân rã dịch vụ và tách biệt cơ sở dữ liệu theo miền dữ liệu:** Sáu thực thể dữ liệu của bài toán thuộc 4 nhóm thông tin khác nhau gồm tài khoản ngân hàng, học phí sinh viên, giao dịch thanh toán và mã xác thực OTP. Việc chia thành 4 dịch vụ đi kèm 4 cơ sở dữ liệu riêng giúp mỗi dịch vụ quản lý dữ liệu của mình, không truy cập trực tiếp vào bảng dữ liệu của dịch vụ khác.

- **Sử dụng API Gateway làm cổng tiếp nhận yêu cầu từ Web Client:** Thay vì để Web Client kết nối trực tiếp tới từng dịch vụ, API Gateway tập trung việc tiếp nhận yêu cầu, kiểm tra xác thực người dùng và ngăn các cuộc gọi từ bên ngoài truy cập trực tiếp vào các thao tác nội bộ như trừ số dư, cập nhật trạng thái học phí hoặc sinh mã OTP.
- Lựa chọn mẫu điều phối Saga tập trung (Orchestration Saga) thay vì Saga phân tán (Choreography Saga): Trong bài toán thanh toán học phí iBanking, luồng nghiệp vụ gồm các bước tuần tự có tính phụ thuộc cao (Kiểm tra nợ → Kiểm tra số dư → Xác thực OTP → Trừ tiền → Gạch nợ → Gửi biên nhận). Việc sử dụng Payment Service làm bộ điều phối tập trung (Orchestrator) mang lại các ưu thế vượt trội: (1) Loại bỏ nguy cơ phụ thuộc vòng (cyclic dependency) giữa Account Service và Tuition Service; (2) Dễ dàng theo dõi, kiểm soát và lưu vết toàn bộ vòng đời giao dịch tại một thực thể máy trạng thái duy nhất; (3) Tinh gọn logic của các dịch vụ tham gia (Account và Tuition chỉ cần cung cấp các API trừ/cộng tiền và gạch nợ độc lập, không cần quan tâm đến ngữ cảnh thanh toán tổng thể).

## 2.4 Microservices Architecture Diagram

Hình 2.1 trình bày Microservices Architecture Diagram của phân hệ đóng học phí iBanking.




*Hình 2.1: Sơ đồ kiến trúc Microservices của phân hệ đóng học phí iBanking*

# CHƯƠNG 3. THIẾT KẾ REST API

## 3.1 Quy chuẩn thiết kế REST API chung của hệ thống

Phân hệ đóng học phí iBanking sử dụng REST/HTTP và định dạng JSON cho giao tiếp giữa Web Client với API Gateway và giữa các microservices nội bộ.

**Hệ thống chuẩn hóa mẫu phản hồi hai trạng thái (success/error) cho các endpoint. Khi xử lý thành công, phản hồi mang trường chỉ thị success mang giá trị true, trường data chứa đối tượng kết quả nghiệp vụ chi tiết, và trường message ghi nhận trạng thái hoàn tất. Khi xuất hiện lỗi nghiệp vụ hoặc sự cố kỹ thuật, phản hồi mang trường chỉ thị success mang giá trị false, đối tượng error bao gồm mã định danh lỗi chuẩn hóa code cùng thông điệp chi tiết message phục vụ công tác gỡ lỗi và thông tin đến người dùng.**

```
// Cấu trúc phản hồi khi xử lý thành công - HTTP 200 OK
{
 "success": true,
 "data": {
  "item_id": 1,
  "status": "COMPLETED"
 },
 "message": "Success"
}

// Cấu trúc phản hồi khi phát sinh lỗi - HTTP 4xx hoặc 5xx
{
 "success": false,
 "error": {
  "code": "ERROR_CODE_IDENTIFIER",
  "message": "Mô tả chi tiết nguyên nhân phát sinh lỗi"
 }
}
```

**Phân cấp phạm vi mạng và cơ chế xác thực quyền hạn:** Hệ thống phân định ranh giới an ninh thành hai tầng mạng độc lập:

- **Tầng giao diện công khai Public API:** Đón nhận yêu cầu từ Web Client qua API Gateway. Người dùng bắt buộc phải đính kèm chuỗi JWT hợp lệ trong tiêu đề HTTP Authorization với định dạng tiền tố Bearer đối với các thao tác cá nhân và thanh toán. Riêng hai endpoint đăng nhập và kiểm tra sức khỏe hệ thống được cấu hình công khai không yêu cầu xác thực.
- **Tầng giao diện nội bộ Internal API:** Hoạt động trong mạng nội bộ, phục vụ trực tiếp các tác vụ điều phối giữa các microservices do Payment Service làm đầu mối Saga. Các yêu cầu nội bộ bắt buộc phải cung cấp khóa bí mật trong tiêu đề HTTP X-Internal-Token. Các yêu cầu trực tiếp từ mạng ngoài đến các endpoint này sẽ bị API Gateway từ chối.

Bảng 3.1: Quy định mã trạng thái HTTP chuẩn của hệ thống

| **Mã HTTP** | **Trạng thái chuẩn** | **Ngữ cảnh áp dụng cụ thể trong hệ thống** |
|---|---|---|
| 200 | OK | Yêu cầu được thực thi hoàn tất. Dữ liệu nghiệp vụ được chứa trong trường data. |
| 400 | Bad Request | Dữ liệu yêu cầu sai cú pháp hoặc vi phạm quy tắc nghiệp vụ như số dư tài khoản không đủ, mã OTP sai, mã số sinh viên sai định dạng, hoặc phiên giao dịch không ở trạng thái chờ. |
| 401 | Unauthorized | Xác thực danh tính thất bại do tên đăng nhập hoặc mật khẩu không chính xác, hoặc thiếu chuỗi JWT Bearer Token hợp lệ trong tiêu đề yêu cầu. |

| Mã HTTP | Trạng thái chuẩn | Ngữ cảnh áp dụng cụ thể trong hệ thống |
|---|---|---|
| 403 | Forbidden | Truy cập bị từ chối do vi phạm quy tắc điều phối luồng nghiệp vụ khi client bên ngoài gọi trực tiếp vào các API nội bộ nhạy cảm. |
| 404 | Not Found | Tài nguyên không tồn tại trong cơ sở dữ liệu như không tìm thấy tài khoản người dùng, mã số sinh viên, khoản học phí hoặc phiên thanh toán. |
| 409 | Conflict | Xung đột trạng thái dữ liệu trong xử lý đồng thời như khoản học phí đã hoàn tất thanh toán trước đó, hoặc một yêu cầu mang cùng Idempotency-Key đang được xử lý. |
| 422 | Unprocessable Entity | Dữ liệu đúng cấu trúc định dạng nhưng nội dung payload gửi lại không khớp với nội dung đã đăng ký trước đó theo Idempotency-Key. |
| 500 | Internal Server Error | Lỗi nội bộ phát sinh từ máy chủ microservice như không thể kết nối hoặc gửi email qua máy chủ SMTP Mailpit. |
| 503 | Service Unavailable | Một hoặc nhiều microservice thành phần tạm thời ngừng hoạt động, được phát hiện qua endpoint kiểm tra sức khỏe. |

## 3.2 Thiết kế REST API của Account Service

Account Service chịu trách nhiệm cung cấp các giao diện lập trình phục vụ xác thực người dùng iBanking và kiểm soát biến động số dư tài khoản. Dịch vụ phân định rõ ràng giữa các endpoint công khai tiếp nhận từ Web Client và các endpoint nội bộ phục vụ điều phối giao dịch thanh toán.

Bảng 3.2: Danh mục REST API của Account Service

| STT | Tên chức năng | Method | URI | Phân loại và xác thực |
|---|---|---|---|---|
| 1 | Đăng nhập hệ thống | POST | /api/accounts/login | Công khai - Không yêu cầu xác thực |
| 2 | Lấy thông tin tài khoản | GET | /api/accounts/me | Công khai - Token Bearer JWT |
| 3 | Khấu trừ số dư tài khoản | POST | /api/accounts/deduct | Nội bộ - Khóa X-Internal-Token |
| 4 | Hoàn tiền số dư tài khoản | POST | /api/accounts/refund | Nội bộ - Khóa X-Internal-Token |

• **API Đăng nhập hệ thống: POST /api/accounts/login**

Mục đích nghiệp vụ: Xác thực thông tin danh tính của người dùng iBanking và cấp phát chuỗi mã xác thực phục vụ các thao tác tài chính tiếp theo.

**Ràng buộc đầu vào:**

• username: Kiểu chuỗi, bắt buộc, độ dài tối thiểu 1 ký tự, không chứa khoảng trắng rỗng.

• password: Kiểu chuỗi, bắt buộc, độ dài tối thiểu 1 ký tự, được băm đối soát bằng bcrypt.

```
// Request Body - POST /api/accounts/login
{
  "username": "sv.nguyen",
  "password": "Test@123"
}

// Response Body - HTTP 200 OK
{
  "success": true,
  "data": {
    "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwidXNlcm5hbWUiOiJzdi5
uZ3V5ZW4iLCJlbWFpbCI6Im5ndXllbi52YW4uYW5Ac3R1ZGVudC50ZHR1LmVkd
S52biJ9.signature",
    "token_type": "bearer"
  },
  "message": "Success"
}

// Error Response - HTTP 401 Unauthorized
{
  "success": false,
  "error": {
    "code": "INVALID_CREDENTIALS",
    "message": "Invalid username or password"
  }
}
```

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 401 INVALID_CREDENTIALS khi thông tin đăng nhập không hợp lệ. Khi xác thực thành công, client nhận được token JWT chứa thông tin tài khoản và thời hạn hiệu lực để đính kèm vào tiêu đề Authorization của các yêu cầu sau.

- **API Lấy thông tin tài khoản người nộp tiền: GET /api/accounts/me**

Mục đích nghiệp vụ: Cung cấp thông tin hồ sơ người dùng cùng số dư tài khoản khả dụng phục vụ đối chiếu công nợ học phí.

**Ràng buộc đầu vào:**

- Tiêu đề Authorization: Kiểu chuỗi, bắt buộc, định dạng Bearer token JWT hợp lệ và chưa hết hạn.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 401 UNAUTHORIZED khi token thiếu hoặc hết hạn, mã lỗi 404 ACCOUNT_NOT_FOUND khi tài khoản không tồn tại. Kết quả thành công cho phép giao diện hiển thị họ tên người nộp và số dư tức thời.

**• API Khấu trừ số dư tài khoản nội bộ: POST /api/accounts/deduct**

Mục đích nghiệp vụ: Thực hiện trừ số dư tài khoản của người nộp tiền trong chuỗi điều phối thanh toán Saga theo yêu cầu từ Payment Service.

**Ràng buộc đầu vào:**

• Tiêu đề X-Internal-Token: Kiểu chuỗi, bắt buộc, khớp với khóa bí mật nội bộ.

• account_id: Kiểu chuỗi, bắt buộc, định danh tài khoản cần trừ tiền.

• amount: Kiểu số thực, bắt buộc, giá trị phải lớn hơn 0 và nhỏ hơn hoặc bằng số dư hiện tại.

• transaction_id: Kiểu chuỗi, bắt buộc, mã giao dịch tham chiếu phục vụ lưu vết đối soát.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 400 INSUFFICIENT_BALANCE khi số dư tài khoản không đủ chi trả. Khi thành công, số dư mới được cập nhật và trả về cho Payment Service làm căn cứ chuyển tiếp sang bước gạch nợ học phí.

**• API Hoàn tiền số dư tài khoản nội bộ: POST /api/accounts/refund**

Mục đích nghiệp vụ: Thực thi thao tác bù trừ hoàn trả số tiền đã khấu trừ khi bước gạch nợ học phí phía sau gặp lỗi, bảo đảm tính nhất quán sau cùng của hệ thống.

**Ràng buộc đầu vào:**

• Tiêu đề X-Internal-Token: Kiểu chuỗi, bắt buộc, khớp với khóa bí mật nội bộ.

• account_id: Kiểu chuỗi, bắt buộc, định danh tài khoản cần hoàn tiền.

• amount: Kiểu số thực, bắt buộc, giá trị phải lớn hơn 0 và đúng bằng số tiền đã trừ trước đó.

• transaction_id: Kiểu chuỗi, bắt buộc, mã giao dịch tham chiếu của bước thanh toán lỗi.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 404 ACCOUNT_NOT_FOUND nếu không tìm thấy tài khoản. Khi thành công, số dư tài khoản được khôi phục nguyên trạng và ghi nhận giao dịch REFUND trong nhật ký đối soát.

## 3.3 Thiết kế REST API của Tuition Service

Tuition Service chịu trách nhiệm cung cấp giao diện quản lý dữ liệu học phí, cho phép tra cứu công nợ sinh viên theo mã số và thực hiện gạch nợ học phí khi nhận được lệnh điều phối hợp lệ.

Bảng 3.3: Danh mục REST API của Tuition Service

| STT | Tên chức năng | Method | URI | Phân loại và xác thực |
|---|---|---|---|---|
| 1 | Tra cứu học phí theo MSSV | GET | /api/tuitions/students/{mssv} | Công khai - Token Bearer JWT |
| 2 | Truy vấn bản ghi học phí | GET | /api/tuitions/records/{tuition_id} | Linh hoạt - Token Bearer hoặc Khóa nội bộ |
| 3 | Gạch nợ học phí nội bộ | POST | /api/tuitions/pay | Nội bộ - Khóa X-Internal-Token |

- **API Tra cứu công nợ học phí sinh viên theo MSSV: GET /api/tuitions/students/{mssv}**

Mục đích nghiệp vụ: Cho phép người nộp tiền tra cứu danh sách các khoản học phí của sinh viên dựa trên mã số sinh viên.

**Ràng buộc đầu vào:**

- Tiêu đề Authorization: Kiểu chuỗi, bắt buộc, định dạng Bearer token JWT hợp lệ.
- Tham số mssv: Kiểu chuỗi trên đường dẫn URI, bắt buộc, độ dài từ 6 đến 10 ký tự chữ và số.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 404 STUDENT_NOT_FOUND khi mã sinh viên không tồn tại trong hệ thống. Khi thành công, trả về thông tin sinh viên kèm mảng các khoản học phí để người dùng lựa chọn khoản nợ cần thanh toán.

- **API Truy vấn bản ghi học phí theo định danh: GET /api/tuitions/records/{tuition_id}**

Mục đích nghiệp vụ: Cung cấp chi tiết một bản ghi học phí cụ thể phục vụ mục đích hiển thị lịch sử và đối soát trạng thái gạch nợ.

**Ràng buộc đầu vào:**

- Tham số tuition_id: Kiểu số nguyên dương trên đường dẫn URI, bắt buộc, là khóa chính của bản ghi học phí.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 404 TUITION_NOT_FOUND nếu không tìm thấy bản ghi. Khi thành công, dữ liệu chi tiết được Payment Service sử dụng để bổ sung thông tin học kỳ trong lịch sử giao dịch.

- **API Gạch nợ học phí nội bộ: POST /api/tuitions/pay**

Mục đích nghiệp vụ: Cập nhật trạng thái khoản học phí thành đã thanh toán PAID sau khi tiền đã được trừ thành công từ tài khoản người nộp.

**Ràng buộc đầu vào:**

- Tiêu đề X-Internal-Token: Kiểu chuỗi, bắt buộc, khớp với khóa bí mật nội bộ.
- tuition_id: Kiểu số nguyên dương, bắt buộc, mã bản ghi học phí cần gạch nợ.
- student_id: Kiểu chuỗi, bắt buộc, mã số sinh viên thụ hưởng.
- semester: Kiểu chuỗi, tùy chọn, tên học kỳ tương ứng.
- amount: Kiểu số thực, bắt buộc, phải khớp chính xác với số tiền nợ quy định của khoản học phí.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 409 TUITION_ALREADY_PAID nếu khoản học phí đã được gạch nợ trước đó, kích hoạt nhánh hoàn tiền bù trừ Saga tại Payment Service. Khi thành công, trạng thái chuyển sang PAID và mốc thời gian hoàn tất được lưu vết.

## 3.4 Thiết kế REST API của Payment Service

Payment Service là đầu mối điều phối nghiệp vụ Saga Orchestrator, kiểm soát toàn bộ chu trình thanh toán học phí từ bước chuẩn bị, xác thực mã bảo mật cho đến bước thực thi thanh toán và truy xuất lịch sử.

Bảng 3.4: Danh mục REST API của Payment Service

| STT | Tên chức năng | Method | URI | Phân loại và xác thực |
|---|---|---|---|---|
| 1 | Khởi tạo thanh toán | POST | /api/payments/initiate | Công khai - Token Bearer JWT |
| 2 | Xác nhận thanh toán OTP | POST | /api/payments/confirm | Công khai - Token Bearer JWT và Idempotency-Key |
| 3 | Truy vấn lịch sử thanh toán | GET | /api/payments/history | Công khai - Token Bearer JWT |

- **API Khởi tạo phiên giao dịch thanh toán học phí: POST /api/payments/initiate**

Mục đích nghiệp vụ: Thiết lập phiên giao dịch ở trạng thái chờ PENDING và kích hoạt quy trình sinh mã xác thực OTP gửi đến hộp thư người dùng.

**Ràng buộc đầu vào:**

- Tiêu đề Authorization: Kiểu chuỗi, bắt buộc, định dạng Bearer token JWT hợp lệ.

• student_id: Kiểu chuỗi, bắt buộc, mã sinh viên cần đóng học phí, độ dài tối thiểu 1 ký tự.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 400 INSUFFICIENT_BALANCE nếu số dư không đủ, hoặc mã lỗi 409 TUITION_ALREADY_PAID nếu học phí đã nộp. Khi hợp lệ, phiên thanh toán PENDING được tạo và trả về payment_id để client chuyển sang màn hình nhập OTP.

- **API Xác nhận giao dịch thanh toán bằng mã OTP: POST /api/payments/confirm**

Mục đích nghiệp vụ: Xác thực mã OTP, chuyển giao trạng thái sang PROCESSING, trừ tiền tại Account Service, gạch nợ tại Tuition Service và gửi email biên nhận.

**Ràng buộc đầu vào:**

• Tiêu đề Authorization: Kiểu chuỗi, bắt buộc, định dạng Bearer token JWT hợp lệ.

• Tiêu đề Idempotency-Key: Kiểu chuỗi chuẩn UUID v4, bắt buộc, phục vụ kiểm soát giao dịch trùng lặp.

• payment_id: Kiểu chuỗi, bắt buộc, mã phiên giao dịch đang ở trạng thái PENDING.

• otp_code: Kiểu chuỗi, bắt buộc, định dạng chính xác 6 chữ số từ 0 đến 9.

```
// Request Headers:
// Authorization: Bearer ACCESS_TOKEN
// Idempotency-Key: 9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d
// Request Body - POST /api/payments/confirm
{
  "payment_id": "1",
  "otp_code": "123456"
}

// Response Body - HTTP 200 OK
{
  "success": true,
  "data": {
    "payment_id": "1",
    "status": "SUCCESS",
    "amount": 4850000.0,
    "tuition_id": 1,
    "student_id": "521H0001",
    "student_name": "Nguyễn Văn An",
    "semester": "HK1/2024-2025",
    "paid_at": "2026-11-09T10:30:00Z"
  },
  "message": "Success"
}

// Error Response - HTTP 400 Bad Request
{
  "success": false,
  "error": {
    "code": "INVALID_OTP",
    "message": "Mã OTP không chính xác. Bạn còn 2 lần thử."
```

```
  }
}
```

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 400 INVALID_OTP khi sai mã OTP, mã lỗi 400 OTP_EXPIRED khi hết hạn 5 phút. Khi bước gạch nợ học phí lỗi, tự động kích hoạt hoàn tiền bù trừ và chuyển trạng thái giao dịch sang FAILED. Khi thành công toàn bộ, giao dịch chuyển sang trạng thái SUCCESS.

- **API Truy vấn lịch sử giao dịch thanh toán của người dùng: GET /api/payments/history**

Mục đích nghiệp vụ: Cung cấp danh sách các giao dịch thanh toán học phí đã thực hiện của tài khoản người nộp tiền phục vụ đối soát và lưu trữ.

**Ràng buộc đầu vào:**

- Tiêu đề Authorization: Kiểu chuỗi, bắt buộc, định dạng Bearer token JWT hợp lệ.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 401 UNAUTHORIZED khi thiếu token. Khi thành công, trả về danh sách giao dịch tổng hợp kèm thông tin sinh viên và học kỳ nhờ cơ chế ghép dữ liệu bất đồng bộ.

## 3.5 Thiết kế REST API của Notification Service

Notification Service đảm nhận trách nhiệm chuyên biệt về quản lý vòng đời mã xác thực bảo mật và gửi thông báo điện tử qua máy chủ SMTP Mailpit. Toàn bộ các API của dịch vụ là API nội bộ, yêu cầu khóa bí mật X-Internal-Token trong tiêu đề HTTP.

Bảng 3.5: Danh mục REST API của Notification Service

| STT | Tên chức năng | Method | URI | Phân loại và xác thực |
|---|---|---|---|---|
| 1 | Sinh và gửi mã OTP | POST | /api/notifications/otp/generate | Nội bộ - Khóa X-Internal-Token |

| STT | Tên chức năng | Method | URI | Phân loại và xác thực |
|---|---|---|---|---|
| 2 | Xác thực mã OTP nội bộ | POST | /api/notifications/otp/verify | Nội bộ - Khóa X-Internal-Token |
| 3 | Gửi email xác nhận thành công | POST | /api/notifications/email/success | Nội bộ - Khóa X-Internal-Token |

- **API Sinh và gửi mã xác thực OTP qua Email: POST /api/notifications/otp/generate**

Mục đích nghiệp vụ: Sinh chuỗi mã xác thực 6 chữ số an toàn gắn với phiên thanh toán và gửi trực tiếp đến hộp thư của người nộp tiền.

**Ràng buộc đầu vào:**

• Tiêu đề X-Internal-Token: Kiểu chuỗi, bắt buộc, khớp với khóa bí mật nội bộ.

• payment_id: Kiểu chuỗi, bắt buộc, mã phiên thanh toán liên kết.

• email: Kiểu chuỗi, bắt buộc, định dạng địa chỉ thư điện tử hợp lệ.

• student_name: Kiểu chuỗi, tùy chọn, họ tên sinh viên thụ hưởng.

• student_id: Kiểu chuỗi, tùy chọn, mã số sinh viên.

• amount: Kiểu số thực, tùy chọn, giá trị lớn hơn 0.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 500 MAIL_DELIVERY_ERROR khi máy chủ SMTP không gửi được thư. Khi thành công, bản ghi OTP được lưu vết với thời hạn hiệu lực 5 phút và tối đa 3 lần thử.

**• API Xác thực mã OTP nội bộ: POST /api/notifications/otp/verify**

Mục đích nghiệp vụ: Đối chiếu mã xác thực do người dùng cung cấp, kiểm tra điều kiện còn hiệu lực và chưa từng bị sử dụng.

**Ràng buộc đầu vào:**

• Tiêu đề X-Internal-Token: Kiểu chuỗi, bắt buộc, khớp với khóa bí mật nội bộ.

• payment_id: Kiểu chuỗi, bắt buộc, mã phiên thanh toán liên kết.

• otp_code: Kiểu chuỗi, bắt buộc, độ dài chính xác 6 chữ số.

Xử lý lỗi và chuyển giao trạng thái: Trả về mã lỗi 400 OTP_EXPIRED khi quá hạn 5 phút, mã lỗi 400 OTP_MAX_ATTEMPTS_EXCEEDED khi nhập sai quá 3 lần. Khi đúng mã, hệ thống đánh dấu mã đã sử dụng để ngăn chặn tấn công phát lại.

- **API Gửi thư điện tử xác nhận thanh toán thành công: POST /api/notifications/email/success**

Mục đích nghiệp vụ: Biên soạn và chuyển phát thư điện tử biên lai thanh toán học phí thành công đến hòm thư người dùng.

**Ràng buộc đầu vào:**

• Tiêu đề X-Internal-Token: Kiểu chuỗi, bắt buộc, khớp với khóa bí mật nội bộ.

• email: Kiểu chuỗi, bắt buộc, địa chỉ hòm thư người nhận.

• payment_id: Kiểu chuỗi, bắt buộc, mã giao dịch đã gạch nợ thành công.

• student_name: Kiểu chuỗi, bắt buộc, họ tên sinh viên.

• student_id: Kiểu chuỗi, bắt buộc, mã số sinh viên.

• amount: Kiểu số thực, bắt buộc, số tiền đã gạch nợ.

• paid_at: Kiểu chuỗi thời gian, bắt buộc, định dạng ISO 8601.

Xử lý lỗi và chuyển giao trạng thái: Trả về trạng thái xác nhận đã gửi thư thành công, hoàn tất toàn bộ chu trình thông báo biên lai đóng học phí.

## 3.6 Cơ chế kiểm soát luồng và bảo vệ API tại API Gateway

API Gateway là điểm tiếp nhận yêu cầu tập trung của hệ thống microservices iBanking, đảm nhiệm định tuyến yêu cầu, kiểm soát quyền truy cập luồng nghiệp vụ, kiểm tra trạng thái hoạt động của các dịch vụ và xử lý chống lặp giao dịch thanh toán.

Bảng 3.6: Quy tắc định tuyến và bộ lọc bảo vệ tại API Gateway

| **Endpoint tiếp nhận tại Gateway** | **Dịch vụ đích** | **Xác thực yêu cầu** | **Bộ lọc bảo vệ áp dụng** |
|---|---|---|---|
| /health | API Gateway - Thăm dò nội bộ | Không yêu cầu | Kiểm tra sức khỏe đồng thời cả 4 microservices |
| /api/accounts/login | Account Service | Không yêu cầu | Chuyển tiếp trực tiếp và ghi nhật ký truy cập |
| /api/accounts/me | Account Service | Token Bearer JWT | Chuyển tiếp tiêu đề Authorization đến Account Service |
| /api/accounts/deduct | Account Service | Chặn truy cập ngoài | Workflow Guard từ chối bằng mã HTTP 403 Forbidden |
| /api/accounts/refund | Account Service | Chặn truy cập ngoài | Workflow Guard từ chối bằng mã HTTP 403 Forbidden |
| /api/tuitions/students/{mssv} | Tuition Service | Token Bearer JWT | Chuyển tiếp tiêu đề Authorization đến Tuition Service |
| /api/tuitions/records/{id} | Tuition Service | Token Bearer JWT | Chuyển tiếp tiêu đề Authorization đến Tuition Service |
| /api/tuitions/pay | Tuition Service | Chặn truy cập ngoài | Workflow Guard từ chối bằng mã HTTP 403 Forbidden |

| Endpoint tiếp nhận tại Gateway | Dịch vụ đích | Xác thực yêu cầu | Bộ lọc bảo vệ áp dụng |
|---|---|---|---|
| /api/payments/initiate | Payment Service | Token Bearer JWT | Chuyển tiếp tiêu đề Authorization đến Payment Service |
| /api/payments/confirm | Payment Service | Token Bearer và Khóa bất biến | Idempotency Middleware khóa luồng và phát lại cache |
| /api/payments/history | Payment Service | Token Bearer JWT | Chuyển tiếp tiêu đề Authorization đến Payment Service |
| /api/notifications/* | Notification Service | Chặn truy cập ngoài | Workflow Guard từ chối bằng mã HTTP 403 Forbidden |

**• Cơ chế kiểm tra sức khỏe hệ thống: Endpoint GET /health**

Mục đích nghiệp vụ: Kiểm tra trạng thái sẵn sàng của từng microservice và phản hồi kết quả tổng hợp cho bộ cân bằng tải.

Gateway sử dụng cơ chế xử lý bất đồng bộ asyncio.gather để gửi các gói tin thăm dò song song đến endpoint kiểm tra sức khỏe của cả bốn microservices thành phần với thời gian chờ tối đa 2.0 giây. Trạng thái tổng thể trả về ok khi tất cả các dịch vụ đều phản hồi mã thành công 200, và chuyển sang trạng thái degraded nếu có bất kỳ dịch vụ nào bị gián đoạn.

**• Cơ chế lọc luồng nghiệp vụ Workflow Guard: Bảo vệ các API nội bộ**

Mục đích nghiệp vụ: Ngăn chặn client bên ngoài gọi trực tiếp vào các chức năng nội bộ nhằm bảo đảm quy trình thanh toán luôn đi qua bộ điều phối Saga.

Gateway duy trì danh sách các route nội bộ gồm accounts deduct, accounts refund, tuitions pay, và các API xử lý OTP của Notification Service. Khi phát hiện yêu cầu từ client bên ngoài gửi tới các route này, Gateway lập tức trả về mã lỗi 403 Forbidden kèm mã WORKFLOW_VIOLATION mà không chuyển tiếp vào mạng nội bộ.

```
// Error Response khi Client ngoài cố tình gọi API nội bộ - HTTP 403 Forbidden
{
  "success": false,
  "error": {
    "code": "WORKFLOW_VIOLATION",
    "message": "Forbidden: Direct access to internal service operation is not
allowed."
  }
}
```

- **Cơ chế kiểm soát trùng lặp yêu cầu thanh toán: Middleware xử lý Idempotency-Key**

Mục đích nghiệp vụ: Ngăn trừ tiền hai lần khi client gửi lặp yêu cầu xác nhận thanh toán do nhấn liên tiếp hoặc retry mạng.

**Cơ chế kỹ thuật qua ba pha xử lý kế tiếp:**

- Khóa luồng đang xử lý In-flight Locking: Gateway thiết lập khóa tổ hợp gồm phương thức, tên dịch vụ, đường dẫn, mã định danh người dùng và chuỗi Idempotency-Key. Khi có hai yêu cầu mang cùng một khóa đến gần như đồng thời, yêu cầu đến sau phát hiện trạng thái đang xử lý IN_PROGRESS sẽ bị chặn lại và nhận mã lỗi 409 Conflict với mã lỗi IDEMPOTENCY_IN_PROGRESS.
- Kiểm tra tính toàn vẹn dữ liệu Payload Hash Validation: Gateway tính toán mã băm SHA-256 từ nội dung gói tin yêu cầu. Nếu một yêu cầu gửi lại mang cùng Idempotency-Key nhưng có nội dung payload bị sửa đổi khác với yêu cầu gốc, Gateway từ chối thực thi và trả về mã lỗi 422 Unprocessable Entity kèm thông báo IDEMPOTENCY_PAYLOAD_MISMATCH.

- Tái phát phản hồi từ bộ nhớ đệm Cached Response Replay: Khi yêu cầu ban đầu hoàn tất thành công, Gateway lưu bản sao kết quả phản hồi vào bộ nhớ đệm với thời gian sống 300 giây. Toàn bộ các yêu cầu gửi lại hợp lệ sau đó sẽ nhận lại kết quả đã lưu kèm tiêu đề HTTP X-Idempotent-Replay mang giá trị true mà không kích hoạt lại các bước trừ tiền hay gạch nợ ở hạ tầng phía sau.

Các cơ chế trên tại API Gateway bảo đảm tính sẵn sàng của hệ thống, kiểm soát quyền thực thi nghiệp vụ và loại trừ rủi ro phát sinh giao dịch trùng lặp.

Hệ thống tích hợp đặc tả OpenAPI 3.0, tự động sinh tài liệu tương tác tại endpoint /docs của API Gateway phục vụ kiểm thử và đối soát nội bộ.

# CHƯƠNG 4. HIỆN THỰC CƠ SỞ DỮ LIỆU

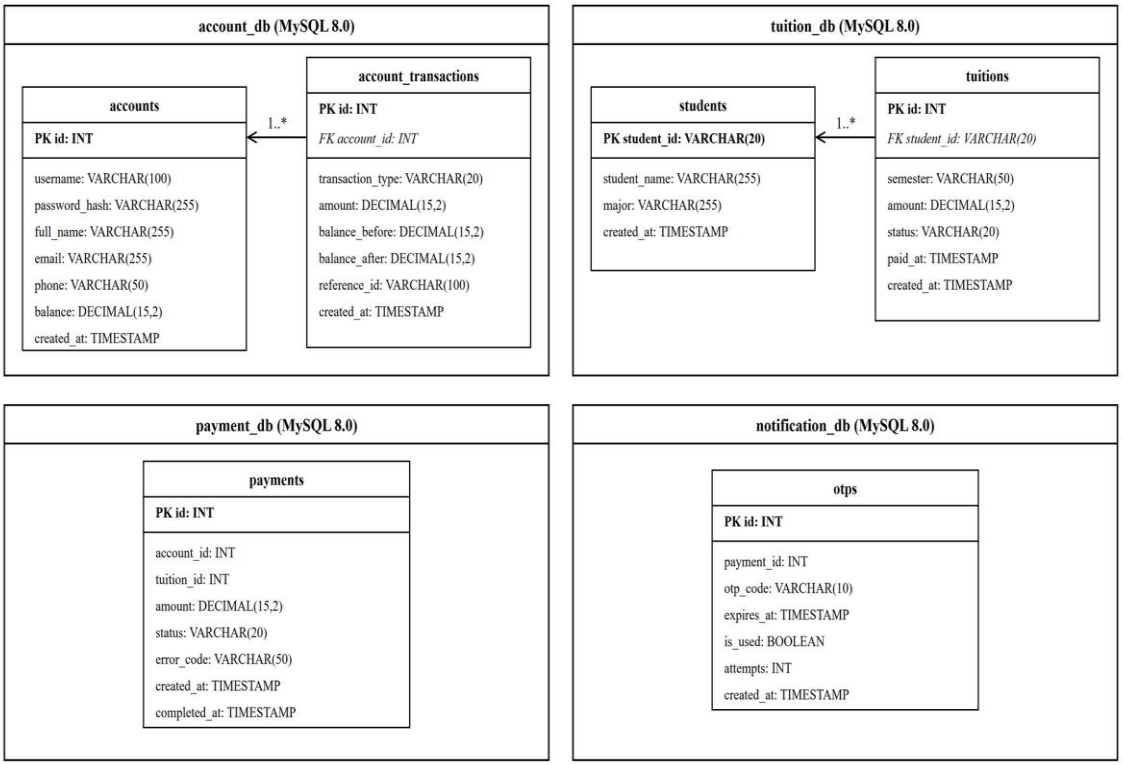
## 4.1 Nguyên tắc hiện thực cơ sở dữ liệu và lưu trữ phân tán

Hệ thống áp dụng mẫu thiết kế Database per Service. Mỗi microservice sở hữu và quản trị một lược đồ cơ sở dữ liệu độc lập trên hệ quản trị MySQL 8.0 với công cụ lưu trữ InnoDB, bảo đảm các tính chất giao dịch ACID trong phạm vi cục bộ của từng dịch vụ.

Nguyên tắc cô lập ranh giới dữ liệu: Mỗi microservice chỉ truy cập trực tiếp lược đồ cơ sở dữ liệu do dịch vụ đó quản lý. Kiến trúc không thiết lập truy vấn kết nối bảng hoặc ràng buộc khóa ngoại vật lý xuyên cơ sở dữ liệu. Mọi thao tác trao đổi dữ liệu giữa các dịch vụ đều được thực hiện qua REST API và mô hình điều phối giao dịch phân tán Saga.

Chuẩn hóa kiểu dữ liệu tiền tệ: Các thuộc tính biểu diễn giá trị tiền tệ gồm số dư tài khoản, số tiền giao dịch và số tiền học phí đều sử dụng kiểu dữ liệu DECIMAL với 15 chữ số và 2 chữ số thập phân. Kiểu dữ liệu này giúp tránh sai số làm tròn số học của kiểu số thực dấu phẩy động, đảm bảo độ chính xác cho các phép tính tài chính.

Hình 4.1 biểu diễn sơ đồ lược đồ cơ sở dữ liệu quan hệ phân tán của bốn Microservices trong phân hệ đóng học phí iBanking, mô tả ranh giới cơ sở dữ liệu độc lập của từng dịch vụ, các bảng dữ liệu thành phần, cấu trúc khóa chính, khóa ngoại cùng các mối quan hệ liên kết bảng nội bộ.




*Hình 4.1: Sơ đồ kiến trúc cơ sở dữ liệu phân tán của các Microservices*

Bảng 4.1: Danh mục cơ sở dữ liệu và thực thể phân tán

| **Tên cơ sở dữ liệu** | **Dịch vụ sở hữu** | **Các bảng dữ liệu** | **Phạm vi trách nhiệm dữ liệu** |
|---|---|---|---|
| account_db | Account Service | accounts, account_transactions | Lưu trữ tài khoản người dùng, mật khẩu băm, số dư khả dụng và nhật ký các lần biến động số dư ghi nợ hoặc hoàn tiền. |
| tuition_db | Tuition Service | students, tuitions | Lưu trữ hồ sơ học tập sinh viên và danh mục các khoản nợ học phí theo từng học kỳ kèm mốc thời gian hoàn tất. |

| Tên cơ sở dữ liệu | Dịch vụ sở hữu | Các bảng dữ liệu | Phạm vi trách nhiệm dữ liệu |
|---|---|---|---|
| payment_db | Payment Service | payments | Lưu trữ trạng thái vòng đời các phiên giao dịch thanh toán học phí phục vụ bộ điều phối Saga và lịch sử người dùng. |
| notification_db | Notification Service | otps | Lưu trữ mã số xác thực bảo mật, thời hạn hiệu lực, số lượt thử còn lại và cờ đánh dấu đã sử dụng. |

## 4.2 Hiện thực cơ sở dữ liệu của Account Service

Cơ sở dữ liệu account_db quản lý thông tin tài khoản và lịch sử biến động số dư của người dùng iBanking. Lược đồ gồm hai bảng quan hệ: bảng accounts lưu trữ hồ sơ tài khoản và bảng account_transactions lưu nhật ký các lần thay đổi số dư phục vụ đối soát.

Bảng accounts quản lý thông tin định danh người dùng với khóa chính id tự tăng và chỉ mục duy nhất trên trường tên đăng nhập username nhằm chống trùng lặp tài khoản. Mật khẩu được mã hóa một chiều bằng giải thuật bcrypt. Thuộc tính số dư tài khoản balance được kiểm soát bởi ràng buộc kiểm tra chk_balance quy định số dư luôn lớn hơn hoặc bằng 0, bảo đảm tài khoản không bị trừ âm ở cấp cơ sở dữ liệu.

Bảng account_transactions ghi nhận các giao dịch trừ tiền hoặc hoàn tiền trên tài khoản với khóa ngoại fk_tx_account tham chiếu đến bảng accounts. Ràng buộc kiểm tra chk_tx_balance_math kiểm soát quan hệ số học giữa số dư trước và sau giao dịch: đối với loại DEDUCT thì số dư sau bằng số dư trước trừ số tiền giao dịch; đối với loại REFUND thì số dư sau bằng số dư trước cộng số tiền giao dịch. Chỉ mục duy

nhất uq_account_ref_type trên bộ ba mã tài khoản, mã tham chiếu và loại giao dịch đảm bảo mỗi giao dịch chỉ tác động số dư một lần duy nhất.

## 4.3 Hiện thực cơ sở dữ liệu của Tuition Service

Cơ sở dữ liệu tuition_db quản trị dữ liệu sinh viên cùng các bản ghi nợ học phí do Tuition Service phụ trách độc lập. Lược đồ được tổ chức thành hai bảng dữ liệu quan hệ gồm bảng students lưu trữ hồ sơ sinh viên và bảng tuitions theo dõi tình trạng công nợ theo từng học kỳ.

Bảng students sử dụng trực tiếp mã số sinh viên student_id làm khóa chính dạng chuỗi ký tự theo chuẩn định danh học vụ của nhà trường, cùng các thông tin họ tên sinh viên và chuyên ngành đào tạo. Bảng tuitions lưu trữ các khoản phải nộp với khóa chính id tự tăng và khóa ngoại fk_tuition_student liên kết trực tiếp với bảng students.

Cơ chế kiểm soát toàn vẹn trên tuition_db chú trọng bảo vệ tính duy nhất và tính nhất quán của trạng thái công nợ. Khóa duy nhất uq_student_semester kết hợp giữa mã số sinh viên và mã học kỳ ngăn chặn việc tạo trùng lặp nhiều khoản nợ cho cùng một sinh viên trong một kỳ học. Ràng buộc toàn vẹn gạch nợ chk_tuition_paid_integrity kiểm soát nghiêm ngặt mối quan hệ giữa trạng thái thanh toán và mốc thời gian hoàn tất. Khi khoản học phí ở trạng thái chưa nộp UNPAID thì mốc thời gian thanh toán bắt buộc phải là giá trị rỗng NULL. Khi khoản nợ đã chuyển sang trạng thái đã nộp PAID thì mốc thời gian thanh toán bắt buộc phải ghi nhận thời điểm hoàn tất hợp lệ. Điều này ngăn ngừa mọi hành vi cập nhật sai lệch trạng thái học phí của sinh viên.

## 4.4 Hiện thực cơ sở dữ liệu của Payment Service

Cơ sở dữ liệu payment_db lưu trữ dữ liệu các phiên giao dịch thanh toán phân tán do Payment Service quản lý. Trung tâm dữ liệu của dịch vụ là bảng payments, nơi ghi nhận toàn bộ vòng đời và tiến trình chuyển đổi của máy trạng thái thanh toán.

Lược đồ bảng payments quản lý mã phiên thanh toán id tự tăng, mã tài khoản account_id, mã công nợ học phí tuition_id, số tiền giao dịch amount, trạng thái xử lý status và mã lỗi chi tiết error_code nếu có sự cố. Các trường account_id và tuition_id không thiết lập khóa ngoại vật lý sang account_db hay tuition_db. Quan hệ liên kết được quản lý ở tầng logic nghiệp vụ và bộ điều phối giao dịch phân tán Saga.

Bảng payments khai thác chỉ mục idx_tuition_id nhằm tăng tốc độ tra cứu các giao dịch gắn với một khoản nợ học phí cụ thể, ngăn ngừa tình huống nhiều tài khoản cùng thanh toán trùng một khoản nợ. Chỉ mục idx_account_created tối ưu hóa tốc độ trích xuất lịch sử giao dịch thanh toán của khách hàng theo thứ tự thời gian mới nhất.

## 4.5 Hiện thực cơ sở dữ liệu của Notification Service

Cơ sở dữ liệu notification_db quản lý dữ liệu vòng đời của các mã xác thực bảo mật một lần do Notification Service phụ trách. Lược đồ tập trung vào bảng otps lưu vết mã số xác thực gắn với từng phiên giao dịch thanh toán nhằm phục vụ cơ chế bảo mật hai yếu tố.

Bảng otps lưu trữ thông tin mã xác thực liên kết với phiên thanh toán payment_id, mã xác thực otp_code, thời điểm hết hạn expires_at, trạng thái sử dụng is_used và số lần thử attempts. Ràng buộc kiểm tra chk_otp_attempts giới hạn số lần thử từ 0 đến 3 lần. Khi vượt quá 3 lần nhập sai, mã xác thực sẽ không còn hiệu lực.

Notification Service khai thác chỉ mục phức hợp idx_verify_otp kết hợp ba thuộc tính payment_id, otp_code và is_used giúp định vị và thẩm định tính hợp lệ của mã xác thực nhanh chóng. Chỉ mục idx_expires_at hỗ trợ kiểm tra điều kiện thời hạn 5 phút và phục vụ dọn dẹp các mã xác thực đã hết hạn. Việc tách biệt kho dữ liệu xác thực giúp Notification Service cô lập dữ liệu nhạy cảm khỏi các dịch vụ khác.

## 4.6 Dữ liệu mẫu khởi tạo và cơ chế lưu trữ bền vững

Hệ thống thiết lập tập dữ liệu mẫu ban đầu thông qua các tệp kịch bản khởi tạo init.sql được tự động thực thi khi khởi chạy cụm máy chủ cơ sở dữ liệu lần đầu. Dữ

liệu mẫu được xây dựng bám sát các kịch bản kiểm thử nghiệp vụ thanh toán học phí thực tế và các tình huống giao dịch đồng thời.

Bảng 4.2: Dữ liệu mẫu tài khoản người dùng trong bảng accounts

| ID | Tên đăng nhập | Họ và tên | Email | Số điện thoại | Số dư ban đầu |
|---|---|---|---|---|---|
| 1 | sv.nguyen | Nguyễn Văn An | nguyen.van.an@student.tdtu.edu.vn | 0901234567 | 7.500.000 VND |
| 2 | sv.tran | Trần Thị Bảo | tran.thi.bao@student.tdtu.edu.vn | 0912345678 | 2.300.000 VND |
| 3 | sv.le | Lê Hoàng Cường | le.hoang.cuong@student.tdtu.edu.vn | 0923456789 | 12.000.000 VND |

Bảng 4.3: Dữ liệu mẫu công nợ học phí sinh viên trong bảng tuitions

| ID | Mã sinh viên | Họ và tên | Học kỳ nộp | Số tiền học phí | Trạng thái nợ |
|---|---|---|---|---|---|
| 1 | 521H0001 | Nguyễn Văn An | HK1/2024-2025 | 4.850.000 VND | UNPAID |
| 2 | 521H0002 | Trần Thị Bảo | HK1/2024-2025 | 3.620.000 VND | UNPAID |
| 3 | 521H0003 | Lê Hoàng Cường | HK1/2024-2025 | 5.190.000 VND | UNPAID |
| 4 | 522H0017 | Phạm Đức Dũng | HK1/2024-2025 | 6.450.000 VND | UNPAID |
| 5 | 522H0041 | Hoàng Thị Yến | HK1/2024-2025 | 3.280.000 VND | UNPAID |

| ID | Mã sinh viên | Họ và tên | Học kỳ nộp | Số tiền học phí | Trạng thái nợ |
|---|---|---|---|---|---|
| 6 | 523H0089 | Võ Minh Khoa | HK1/2024-2025 | 4.720.000 VND | UNPAID |

Cơ chế lưu trữ bền vững Data Persistence: Nhằm duy trì dữ liệu khi container dừng hoặc khởi động lại, hệ thống sử dụng vùng lưu trữ Docker Named Volumes gắn kết với thư mục /var/lib/mysql của máy chủ cơ sở dữ liệu MySQL 8.0.

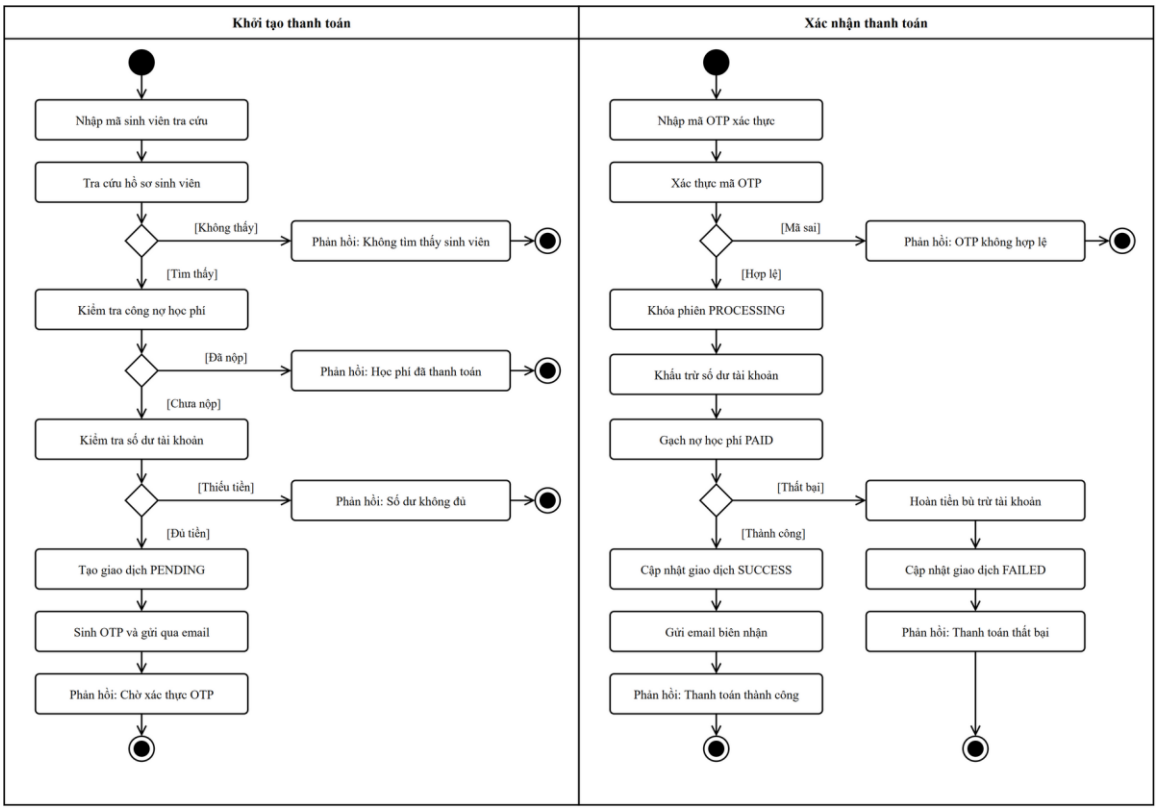
Cơ chế này tách biệt dữ liệu của bốn cơ sở dữ liệu: account_db_data lưu thông tin tài khoản và giao dịch, tuition_db_data lưu hồ sơ sinh viên và công nợ học phí, payment_db_data lưu các phiên thanh toán và notification_db_data lưu dữ liệu mã OTP. Dữ liệu được bảo toàn nguyên vẹn qua các chu kỳ khởi động lại container.

# CHƯƠNG 5. HIỆN THỰC QUY TRÌNH THANH TOÁN

## 5.1 Chức năng đăng nhập, thông tin tài khoản và tra cứu học phí

Quy trình thanh toán trực tuyến yêu cầu dữ liệu người nộp từ Account Service và dữ liệu học phí từ Tuition Service. Hai phân hệ này hoạt động độc lập và giao tiếp qua API Gateway.

Hình 5.1 biểu diễn sơ đồ quy trình nghiệp vụ thanh toán học phí hai pha kết hợp xác thực mã OTP và cơ chế bù trừ Saga. Pha thứ nhất bao gồm các bước tra cứu hồ sơ sinh viên, kiểm tra công nợ học phí, kiểm tra số dư tài khoản, khởi tạo phiên thanh toán PENDING và phát hành mã xác thực qua thư điện tử. Pha thứ hai thực thi điều phối giao dịch phân tán, xác thực OTP kèm khóa phiên PROCESSING, khấu trừ tiền tài khoản, gạch nợ học phí và phát hành biên nhận điện tử hoặc kích hoạt nhánh hoàn trả số dư bù trừ Saga khi phát sinh xung đột.




*Hình 5.1: Sơ đồ quy trình thanh toán học phí hai pha và xác thực OTP*

Hiện thực xác thực danh tính và phát hành thẻ bài truy cập tại Account Service: Quy trình xác thực đăng nhập được xử lý tại hàm login trong tệp backend/account-service/app/accounts/service.py. Khi người dùng gửi tên đăng nhập và mật khẩu, dịch vụ tìm kiếm bản ghi tài khoản tương ứng và sử dụng hàm bcrypt.checkpw để đối chiếu mật khẩu gửi lên với chuỗi băm lưu trữ trong cơ sở dữ liệu. Sau khi thông tin được chứng thực, hệ thống phát hành thẻ bài truy cập JWT ký bằng thuật toán đối xứng HS256 với thời hạn hiệu lực được cấu hình qua biến môi trường. Khi ứng dụng gọi yêu cầu lấy thông tin cá nhân, hàm get_profile trích xuất dữ liệu từ bảng accounts và trả về các trường bắt buộc gồm họ và tên, số điện thoại, địa chỉ email cùng số dư khả dụng hiện tại. Các thông tin định danh này được cố định tự động từ phiên làm việc và người nộp tiền không thể chỉnh sửa.

Hiện thực tra cứu công nợ học phí sinh viên tại Tuition Service: Nghiệp vụ tra cứu học phí được xử lý bởi hàm get_tuition trong tệp backend/tuition-service/app/tuitions/service.py. Chuỗi mã số sinh viên đầu vào được chuẩn hóa bằng cách loại bỏ khoảng trắng thừa và chuyển toàn bộ sang ký tự in hoa. Hệ thống áp dụng biểu thức chính quy kiểm tra định dạng nhằm bảo đảm mã số có độ dài từ 6 đến 10 ký tự chữ và số hợp lệ. Tiếp theo, hệ thống truy vấn cơ sở dữ liệu để lấy thông tin hồ sơ sinh viên và bản ghi công nợ tương ứng. Khi mã sinh viên không tồn tại trong cơ sở dữ liệu, dịch vụ lập tức trả về mã lỗi 404 kèm thông báo không tìm thấy sinh viên. Phân hệ đóng học phí chỉ cho phép thanh toán toàn bộ khoản học phí của học kỳ và không cho phép thanh toán từng phần.

## 5.2 Khởi tạo giao dịch, kiểm tra số dư và trạng thái học phí

Payment Service kiểm tra tuần tự trạng thái học phí và số dư tài khoản trước khi khởi tạo phiên giao dịch.

Quy trình kiểm tra chéo điều kiện và khởi tạo phiên giao dịch: Khi nhận yêu cầu khởi tạo từ phía người dùng, hàm initiate trong tệp payment_service.py thực hiện bốn bước tuần tự:

- **Bước 1 - Kiểm tra trạng thái khoản nợ học phí:** Payment Service gọi giao diện nội bộ sang Tuition Service để tra cứu công nợ của sinh viên. Nếu khoản học phí đã mang trạng thái PAID, hệ thống lập tức từ chối yêu cầu và phản hồi mã lỗi TUITION_ALREADY_PAID kèm mã trạng thái 409.
- **Bước 2 - Kiểm tra khả năng chi trả của số dư tài khoản:** Payment Service gọi giao diện sang Account Service để trích xuất số dư khả dụng thực tế của người nộp. Nếu số dư nhỏ hơn số tiền học phí cần đóng, hệ thống từ chối khởi tạo và phản hồi mã lỗi INSUFFICIENT_BALANCE kèm mã trạng thái 400.
- **Bước 3 - Thiết lập bản ghi giao dịch chờ:** Sau khi vượt qua hai điều kiện kiểm tra trên, Payment Service khởi tạo một bản ghi thanh toán mới trong bảng payments với trạng thái PENDING.
- Bước 4 - Kích hoạt phát hành mã xác thực: Payment Service gửi yêu cầu sang Notification Service để sinh mã xác thực và chuyển phát thư điện tử đến hòm thư của người nộp tiền. Nếu quá trình gửi thông báo gặp sự cố, phiên thanh toán được chuyển sang trạng thái FAILED nhằm đảm bảo tính nhất quán dữ liệu (hiện thực tại hàm initiate trong tệp backend/payment-service/app/payments/service.py).

## 5.3 Quy trình sinh mã OTP, kiểm soát thời hạn và gửi email

Notification Service phụ trách sinh mã OTP 6 chữ số, quản lý thời hạn 5 phút, giới hạn 3 lần thử và gửi thư điện tử thông báo.

Các quy tắc bảo mật và hiện thực vòng đời mã xác thực tại Notification Service:

- Thuật toán sinh mã số ngẫu nhiên an toàn và đảm bảo tính duy nhất: Hàm generate_otp sử dụng thư viện secrets tạo chuỗi 6 chữ số ngẫu nhiên đạt chuẩn an toàn mật mã. Mã sinh ra được kiểm tra thông qua hàm is_otp_active nhằm bảo đảm không trùng lặp với các mã xác thực đang còn hiệu lực của các phiên giao dịch khác.

- **Quản lý thời hạn hiệu lực tối đa 5 phút:** Mỗi bản ghi mã xác thực trong bảng otps được gán mốc thời gian hết hạn sau 300 giây kể từ thời điểm khởi tạo. Khi người dùng xác thực, hệ thống so khớp thời gian thực tế với mốc hết hạn này và từ chối các mã đã quá hạn với mã lỗi OTP_EXPIRED.
- **Cơ chế sử dụng một lần và giới hạn tối đa 3 lần thử:** Hàm verify_otp quản lý số lượt thử sai của người dùng. Khi người dùng nhập sai, trường attempts tăng thêm 1; nếu số lần thử vượt quá 3 lần, mã xác thực bị khóa vĩnh viễn với mã lỗi OTP_MAX_ATTEMPTS_EXCEEDED. Khi người dùng nhập đúng mã, cờ is_used lập tức được chuyển thành TRUE để loại trừ nguy cơ tái sử dụng mã xác thực.
- Gửi email thông báo qua giao thức mạng chuyển thư: Notification Service sử dụng thư viện kết nối chuẩn SMTP gửi thư điện tử chứa mã xác thực, mã phiên thanh toán, tên sinh viên và số tiền cần nộp đến hộp thư của người nộp trên máy chủ Mailpit (hiện thực tại hàm generate_otp và verify_otp trong tệp backend/notification-service/app/notifications/service.py).

## 5.4 Hoàn tất thanh toán, gạch nợ và gửi biên nhận xác nhận

Sau khi mã xác thực được kiểm tra hợp lệ, Payment Service điều phối quy trình khấu trừ số dư tài khoản, gạch nợ học phí, ghi nhận lịch sử giao dịch và gửi thư điện tử biên nhận.

Hàm confirm thực hiện các bước sau:

- **Bước 1 - Xác thực mã và khóa phiên xử lý:** Payment Service kiểm tra mã xác thực thông qua hàm verify_otp_and_lock_processing, đồng thời chuyển trạng thái phiên giao dịch từ PENDING sang PROCESSING nhằm khóa phiên, ngăn chặn các thao tác gửi trùng lặp.
- **Bước 2 - Khấu trừ số dư tài khoản người nộp:** Payment Service gọi giao diện nội bộ của Account Service để trừ số tiền tương ứng khỏi tài khoản người nộp và ghi nhận dòng nhật ký khấu trừ vào sổ cái account_transactions.

- **Bước 3 - Gạch nợ học phí sinh viên:** Payment Service gọi giao diện của Tuition Service để chuyển trạng thái khoản học phí từ UNPAID sang PAID kèm mốc thời gian hoàn tất giao dịch.
- **Bước 4 - Cập nhật trạng thái thành công và lưu lịch sử:** Khi các bước cập nhật dữ liệu hoàn thành, Payment Service chuyển trạng thái giao dịch sang SUCCESS và ghi nhận thời điểm hoàn tất completed_at. Dữ liệu này được lưu trữ bền vững phục vụ chức năng tra cứu lịch sử giao dịch qua hàm history.
- Bước 5 - Phát hành thư điện tử biên nhận xác nhận: Payment Service gọi Notification Service kích hoạt hàm send_success_email để gửi thư biên nhận thành công gồm mã giao dịch, tên sinh viên, số tiền và thời gian thanh toán đến hòm thư người nộp tiền, đồng thời trả dữ liệu kết quả hiển thị lên giao diện người dùng (hiện thực luồng Saga hai pha tại hàm confirm trong tệp backend/payment-service/app/payments/service.py).

Bảng 5.1: Ma trận kiểm soát điều kiện và xử lý ngoại lệ trong quy trình thanh toán

| Giai đoạn | Bước nghiệp vụ | Điều kiện kiểm tra | Dịch vụ thực thi | Mã lỗi hoặc Trạng thái | Hành động xử lý của hệ thống |
|---|---|---|---|---|---|
| Pha 1 | Tra cứu học phí | Định dạng MSSV không hợp lệ | Tuition Service | 400 INVALID_MSSV_FORMAT | Từ chối tra cứu, yêu cầu người dùng kiểm tra lại mã sinh viên |
| Pha 1 | Tra cứu học phí | Mã sinh viên không tồn tại | Tuition Service | 404 STUDENT_NOT_FOUND | Phản hồi không tìm thấy hồ sơ sinh viên trong hệ thống |
| Pha 1 | Kiểm tra công nợ | Khoản nợ đã nộp trước đó | Payment Service | 409 TUITION_ALREADY_PAID | Chặn khởi tạo phiên, thông báo học phí đã được thanh toán |
| Pha 1 | Kiểm tra số dư | Số dư khả dụng nhỏ hơn học phí | Payment Service | 400 INSUFFICIENT_BALANCE | Chặn khởi tạo phiên, yêu cầu nạp thêm tiền vào tài khoản |
| Pha 1 | Phát hành OTP | Học phí chưa nộp và số dư đủ | Payment Service | 200 PENDING | Khởi tạo bản ghi PENDING, sinh OTP 6 số và gửi qua Mailpit |

| Giai đoạn | Bước nghiệp vụ | Điều kiện kiểm tra | Dịch vụ thực thi | Mã lỗi hoặc Trạng thái | Hành động xử lý của hệ thống |
|---|---|---|---|---|---|
| Pha 2 | Kiểm tra mã OTP | Mã xác thực không chính xác | Notification Service | 400 INVALID_OTP | Tăng biến đếm attempts, cho phép thử lại nếu chưa quá 3 lần |
| Pha 2 | Kiểm tra lượt thử | Nhập sai quá 3 lần | Notification Service | 400 OTP_MAX_ATTEMPTS_EXCEEDED | Khóa vĩnh viễn mã OTP, hủy phiên giao dịch |
| Pha 2 | Kiểm tra thời hạn | Mã OTP đã quá 5 phút | Notification Service | 400 OTP_EXPIRED | Từ chối xác thực do mã bảo mật đã hết hạn hiệu lực |
| Pha 2 | Hoàn tất giao dịch | Mã OTP chính xác và còn hạn | Payment Service | 200 SUCCESS | Khóa PROCESSING, trừ tiền, gạch nợ PAID, gửi email biên nhận |
| Pha 2 | Xử lý xung đột | Gạch nợ học phí thất bại do tranh chấp đồng thời | Payment Service | 409 PAYMENT_FAILED (DB: TUITION_MARK_FAILED) | Kích hoạt bù trừ Saga hoàn trả số dư tài khoản và cập nhật FAILED |

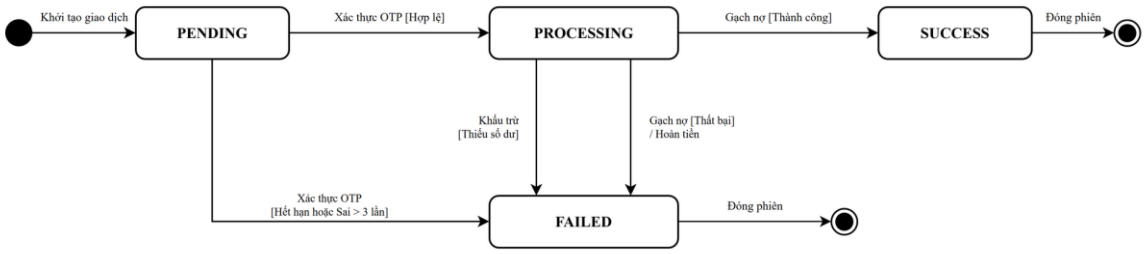
# CHƯƠNG 6. XỬ LÝ GIAO DỊCH VÀ ĐỒNG THỜI

## 6.1 Đảm bảo tính nhất quán dữ liệu giữa các dịch vụ

Trong kiến trúc hướng dịch vụ với bốn cơ sở dữ liệu vật lý riêng biệt, một quy trình thanh toán học phí đòi hỏi sự phối hợp cập nhật dữ liệu trên ba dịch vụ gồm Payment Service, Account Service và Tuition Service. Do các phân hệ hoạt động trên các tiến trình độc lập, hệ thống không thể sử dụng giao thức khóa hai pha truyền thống vì nguy cơ nghẽn mạng và vi phạm tính tự chủ dữ liệu. Thay vào đó, giải pháp được lựa chọn là mô hình Saga điều phối trung tâm với Payment Service điều phối các giao dịch cục bộ và kích hoạt bù trừ hoàn tiền khi phát sinh sự cố.

**Máy trạng thái giao dịch và nguyên lý chuyển dịch trạng thái:** Vòng đời của mỗi phiên thanh toán được mô hình hóa thành một máy trạng thái hữu hạn kiểm soát nghiêm ngặt tại cơ sở dữ liệu payment_db. Mọi thao tác cập nhật trạng thái đều được thực thi nguyên tử thông qua các câu lệnh cập nhật có điều kiện nhằm ngăn chặn bước nhảy trạng thái trái phép.

Hình 6.1 minh họa chi tiết cấu trúc máy trạng thái giao dịch và các nhánh chuyển dịch trạng thái bù trừ Saga trong phân hệ thanh toán học phí iBanking.

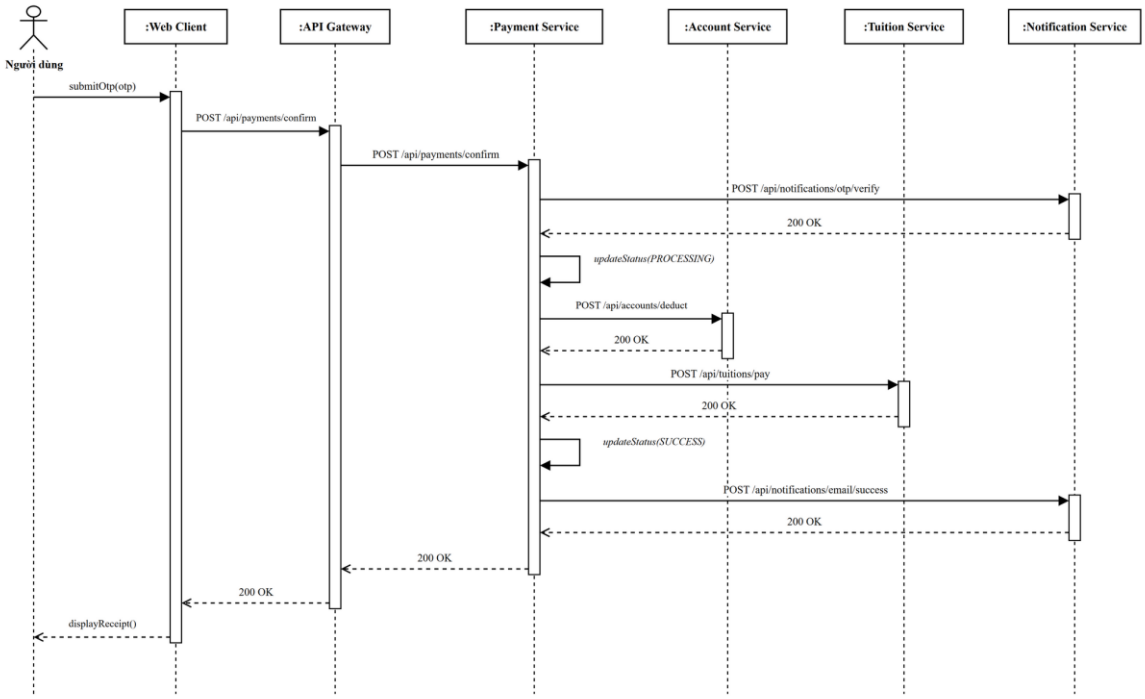



*Hình 6.1: Sơ đồ máy trạng thái giao dịch và cơ chế bù trừ Saga*

Luồng tương tác tuần tự trong kịch bản thanh toán thuận lợi: Khi người dùng nhấn nút xác nhận (sự kiện submitOtp), Web Client gửi yêu cầu xác nhận POST /api/payments/confirm qua API Gateway tới Payment Service. Đóng vai trò bộ điều phối Saga (Orchestrator), Payment Service lần lượt thực thi các bước cục bộ: xác thực mã OTP qua dịch vụ thông báo (POST /api/notifications/otp/verify), cập nhật trạng

thái nội bộ sang PROCESSING (hàm updateStatus), gọi Account Service để khấu trừ số dư (POST /api/accounts/deduct), và gọi Tuition Service để thực hiện gạch nợ học phí (POST /api/tuitions/pay). Khi tất cả các bước hạ tầng đều phản hồi thành công (mã trạng thái HTTP 200 OK), Payment Service chốt trạng thái giao dịch sang SUCCESS, gọi Notification Service gửi email biên nhận (POST /api/notifications/email/success), đồng thời trả kết quả 200 OK về máy khách để hiển thị biên nhận hoàn tất lên màn hình.

Hình 6.2 thể hiện biểu đồ tuần tự các bước tương tác giữa người dùng và các dịch vụ thành phần trong luồng thanh toán học phí thành công.




*Hình 6.2: Biểu đồ tuần tự luồng thanh toán học phí thành công*

Mô hình Saga điều phối bảo đảm hệ thống đạt trạng thái nhất quán sau cùng và duy trì tính sẵn sàng của các dịch vụ phân tán.

## 6.2 Xử lý nhiều giao dịch đồng thời trên cùng một tài khoản

Tình huống tranh chấp đồng thời trên cùng một tài khoản xảy ra khi chủ tài khoản hoặc các tiến trình song song phát sinh nhiều giao dịch thanh toán cùng lúc. Nếu hệ thống chỉ áp dụng thao tác đọc và kiểm tra số dư thông thường mà không có cơ chế cô lập, cả hai giao dịch đều ghi nhận số dư ban đầu đủ khả năng chi trả, dẫn đến việc cả hai cùng được phê duyệt và làm số dư tài khoản bị âm, vi phạm nghiêm trọng nguyên tắc kế toán ngân hàng.

**Giải pháp kỹ thuật phòng vệ bốn lớp tại phân hệ Account Service: Để loại trừ nguy cơ chi tiêu vượt quá số dư khả dụng, hàm deduct_balance trong tệp account_repository.py hiện thực cơ chế bảo vệ bốn lớp khép kín:**

- **Lớp 1 - Kiểm tra tính bất biến trên sổ cái tài chính:** Trước khi xử lý, hệ thống kiểm tra bảng account_transactions để xác minh mã phiên giao dịch này đã được khấu trừ trước đó hay chưa. Nếu đã tồn tại bản ghi khấu trừ hợp lệ, hệ thống tái sử dụng số dư kết quả mà không thực hiện trừ tiền lần hai.
- **Lớp 2 - Khóa bi quan mức dòng dữ liệu:** Hệ thống thực thi câu lệnh truy vấn có khóa dòng SELECT balance FROM accounts WHERE id = %s FOR UPDATE trong một giao dịch MySQL InnoDB tường minh. Khi giao dịch đầu tiên chiếm giữ khóa độc quyền trên dòng tài khoản, mọi giao dịch đồng thời khác bắt buộc phải xếp hàng chờ tại tầng cơ sở dữ liệu cho đến khi giao dịch đầu tiên hoàn tất commit hoặc rollback.
- **Lớp 3 - Cập nhật nguyên tử có điều kiện bảo vệ:** Câu lệnh cập nhật số dư được gắn điều kiện kiểm tra nghiêm ngặt UPDATE accounts SET balance = balance - %s WHERE id = %s AND balance >= %s. Điều kiện này bảo đảm ngay cả trong tình huống hy hữu có sự can thiệp ngoài luồng, số dư hiện tại bắt buộc phải lớn hơn hoặc bằng số tiền cần trừ thì thao tác mới có hiệu lực.
- **Lớp 4 - Ràng buộc toàn vẹn mức hệ quản trị:** Ràng buộc chk_balance đã thiết lập tại lược đồ cơ sở dữ liệu là chốt chặn kỹ thuật cuối cùng, từ chối bất kỳ câu lệnh nào làm giá trị cột balance nhỏ hơn 0.

Nhờ cơ chế bảo vệ bốn lớp này, giao dịch đến trước được phê duyệt và khấu trừ chính xác, trong khi giao dịch đến sau khi giải phóng khóa đọc thấy số dư mới không đủ sẽ bị từ chối an toàn với mã lỗi INSUFFICIENT_BALANCE, loại trừ rủi ro chi tiêu vượt quá hạn mức.

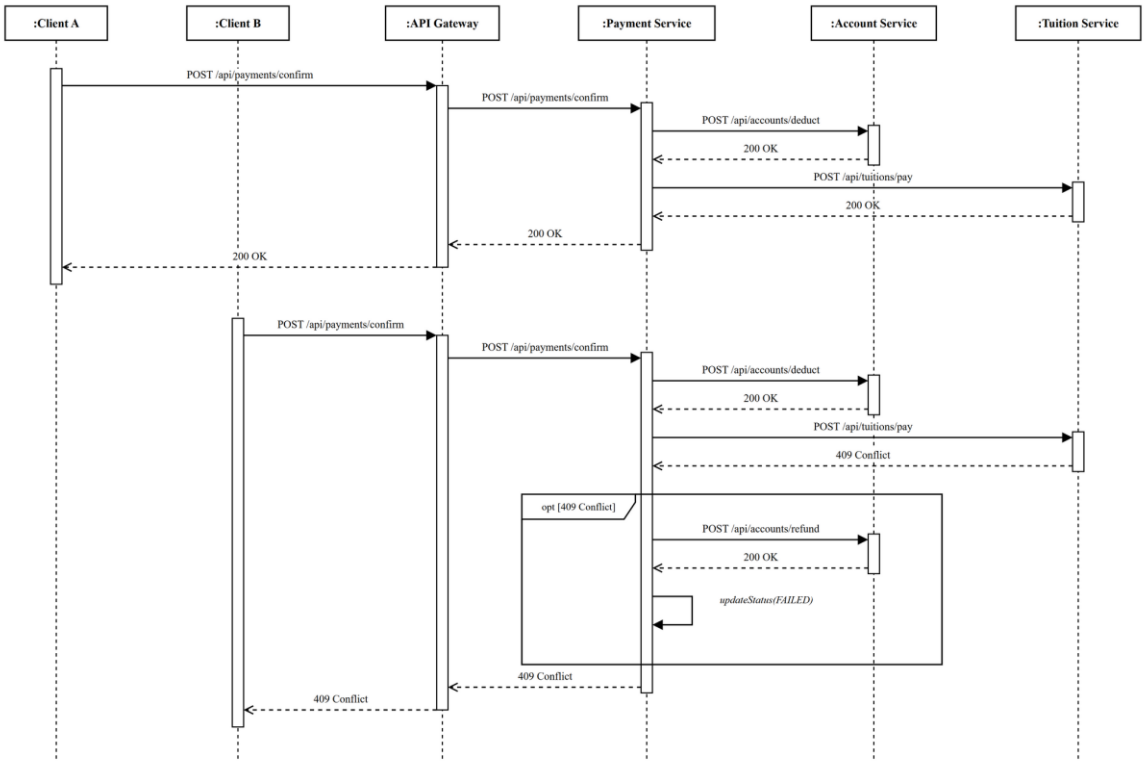
## 6.3 Xử lý nhiều tài khoản cùng thanh toán một khoản học phí

Tình huống hai người dùng khác nhau cùng tra cứu và thanh toán học phí cho cùng một sinh viên là kịch bản tranh chấp liên dịch vụ phức tạp nhất trong hệ thống. Cả hai tài khoản đều sở hữu số dư hợp lệ và cùng vượt qua bước xác thực mã OTP. Quy tắc nghiệp vụ bắt buộc là khoản học phí chỉ được phép ghi nhận thanh toán thành công đúng một lần duy nhất, đồng thời tài khoản bị gạch nợ chậm hơn được hoàn trả lại tiền tự động, chính xác và minh bạch.

**Cơ chế phối hợp giữa gạch nợ nguyên tử và giao dịch bù trừ Saga: Hệ thống giải quyết bài toán này thông qua sự kết hợp giữa chốt chặn khóa dòng tại Tuition Service và quy trình bù trừ tự động tại Payment Service:**

- **Bước gạch nợ nguyên tử tại phân hệ Tuition Service:** Hàm mark_paid trong tệp tuition_repository.py thực hiện câu lệnh SELECT id, amount, status FROM tuitions WHERE id = %s FOR UPDATE. Khi hai yêu cầu gạch nợ đồng thời gửi tới, phiên xử lý đầu tiên chiếm khóa dòng và kiểm tra điều kiện status bằng UNPAID. Sau khi xác thực hợp lệ, dịch vụ thực thi cập nhật trạng thái PAID kèm mốc thời gian hoàn tất vào bản ghi học phí tương ứng. Khi phiên thứ hai được cấp quyền truy cập, trạng thái bản ghi lúc này đã là PAID nên điều kiện kiểm tra không còn thỏa mãn, hệ thống lập tức rollback và phản hồi mã lỗi TUITION_ALREADY_PAID kèm mã trạng thái 409 Conflict.
- Kích hoạt giao dịch bù trừ hoàn tiền Saga tại Payment Service: Do phiên giao dịch thứ hai đã khấu trừ tiền của người nộp tại Account Service trước khi gọi sang Tuition Service, khi nhận được mã lỗi phản hồi 409 Conflict từ Cổng trường, khối điều kiện opt [409 Conflict] tại Payment Service lập tức kích hoạt chu trình bù trừ. Dịch vụ gửi yêu cầu hoàn tiền POST /api/accounts/refund tới Account Service để khôi phục lại số dư đã khấu trừ, ghi nhận bút toán

REFUND vào sổ cái kế toán, chuyển trạng thái phiên giao dịch nội bộ sang FAILED (hàm updateStatus), và phản hồi mã lỗi 409 Conflict ngược lại cho Client B qua API Gateway nhằm thông báo khoản nợ học phí đã được đóng và số dư tài khoản đã được hoàn trả an toàn.




*Hình 6.3: Biểu đồ tuần tự xử lý xung đột đồng thời và bù trừ hoàn tiền Saga*

Cơ chế khóa dòng kết hợp nhánh bù trừ Saga (hiện thực tại hàm mark_paid trong tệp backend/tuition-service/app/tuitions/repository.py) bảo đảm khoản nợ học phí chỉ được gạch nợ một lần và số dư của người nộp sau được hoàn trả tự động.

## 6.4 Cơ chế chống gửi trùng yêu cầu bằng Idempotency-Key tại API Gateway

Nhằm nâng cao chất lượng kỹ thuật của hệ thống theo tiêu chuẩn chấm điểm tối đa, phân hệ API Gateway được trang bị cơ chế kiểm soát bất biến thông qua tiêu đề Idempotency-Key. Cơ chế này ngăn chặn hữu hiệu hành vi gửi trùng yêu cầu thanh

toán do người dùng thao tác nhấp chuột liên tiếp hoặc do mạng truyền thông tự động thử lại yêu cầu.

**Nguyên lý hoạt động của bộ lưu trữ khóa bất biến tại API Gateway:** Bộ nhớ đệm IdempotencyStore trong tệp idempotency.py quản lý trạng thái của các yêu cầu gửi đến cổng trung tâm dựa trên bốn kỹ thuật then chốt:

- Phân tách phạm vi khóa theo người dùng: Khóa bất biến được tạo lập bằng cách kết hợp chuỗi tiêu đề Idempotency-Key do máy khách gửi lên với mã định danh người dùng trích xuất từ thẻ bài JWT. Thiết kế này ngăn ngừa xung đột khóa giữa các tài khoản người dùng khác nhau.
- **Khóa đồng bộ các yêu cầu đang thực thi:** Khi yêu cầu đầu tiên đến, hệ thống khởi tạo bản ghi với trạng thái IN_PROGRESS kèm một đối tượng đồng bộ asyncio.Event. Nếu một yêu cầu trùng lặp gửi đến trong lúc yêu cầu đầu tiên đang được xử lý, tiến trình này sẽ tạm dừng chờ tín hiệu từ đối tượng đồng bộ thay vì chuyển tiếp yêu cầu xuống các dịch vụ backend.
- **Phản hồi từ bộ nhớ đệm kết quả:** Khi yêu cầu gốc xử lý thành công, kết quả phản hồi và mã trạng thái HTTP được lưu vào bộ nhớ đệm với thời gian sống 300 giây. Các yêu cầu gửi trùng sau đó được hoàn trả ngay kết quả lưu sẵn, bảo vệ dịch vụ backend khỏi áp lực quá tải.
- Giải phóng khóa khi phát sinh lỗi nghiệp vụ: Khi yêu cầu thất bại do nhập sai mã xác thực, hàm release_failed xóa khóa khỏi bộ nhớ đệm, cho phép người dùng thực hiện lại thao tác xác nhận (hiện thực tại lớp IdempotencyStore trong tệp api-gateway/app/middlewares/idempotency.py).

# CHƯƠNG 7. HIỆN THỰC GIAO DIỆN NGƯỜI DÙNG

## 7.1 Cấu trúc ứng dụng Web và cơ chế tích hợp với Backend API

Giao diện người dùng của phân hệ đóng học phí được xây dựng dưới dạng ứng dụng Web đơn trang Single Page Application trên nền tảng React 18, kết hợp công cụ đóng gói Vite và bộ khung giao diện tiện ích TailwindCSS. Ứng dụng là điểm tiếp xúc duy nhất của người dùng, kết nối đồng bộ tới các microservices qua đầu mối trung tâm API Gateway.

**Tổ chức cấu trúc phân lớp mã nguồn giao diện:** Mã nguồn ứng dụng tại thư mục frontend được cấu trúc theo mô hình phân tách mối quan tâm rõ rệt thành bốn tầng chuyên biệt:

- **Tầng quản lý phiên xác thực toàn cục AuthContext:** Lưu trữ mã thông báo truy cập JWT và thông tin hồ sơ người dùng trong bộ nhớ trạng thái React và đồng bộ vào kho lưu trữ cục bộ của trình duyệt. Cung cấp hàm đồng bộ số dư để tự động cập nhật số dư khả dụng mới nhất từ dịch vụ Account Service sau mỗi giao dịch thanh toán thành công.
- **Tầng dịch vụ giao tiếp mạng và đánh chặn yêu cầu:** Sử dụng thư viện Axios thiết lập cấu hình trung tâm kết nối về địa chỉ cổng API Gateway. Cơ chế Request Interceptor tự động trích xuất mã token từ bộ nhớ để gắn vào tiêu đề Authorization Bearer trong mọi yêu cầu gửi đi. Cơ chế Response Interceptor lắng nghe phản hồi và tự động điều hướng người dùng về trang đăng nhập khi phát hiện mã lỗi 401 Unauthorized do phiên làm việc hết hạn.
- **Tầng dịch vụ nghiệp vụ chuyên trách:** Đóng gói các hàm gọi API thành các dịch vụ độc lập gồm dịch vụ xác thực tài khoản, dịch vụ tra cứu học phí và dịch vụ thanh toán. Đặc biệt, hàm xác nhận thanh toán tự động sinh và đính kèm tiêu đề Idempotency-Key để bảo vệ giao dịch khỏi nguy cơ gửi trùng lặp.
- Tầng thành phần giao diện và các trang tương tác: Bao gồm trang đăng nhập, trang tra cứu thanh toán học phí, trang lịch sử giao dịch và các hộp thoại chức năng như hộp thoại nhập mã OTP và hộp thoại hiển thị biên lai hoàn tất (tầng

giao tiếp mạng được cấu hình với Axios Interceptor tại tệp frontend/src/lib/api.js để tự động đính kèm mã xác thực JWT Bearer và điều hướng phiên làm việc).

Để bảo đảm tính nhất quán trong luồng tương tác và hỗ trợ việc đối soát kiến trúc giữa các tầng phân tán, Bảng 7.1 tổng hợp chi tiết mối liên kết ánh xạ giữa các hành vi tương tác trên giao diện người dùng, các điểm cuối REST API đi qua API Gateway và các microservices nghiệp vụ xử lý tại backend:

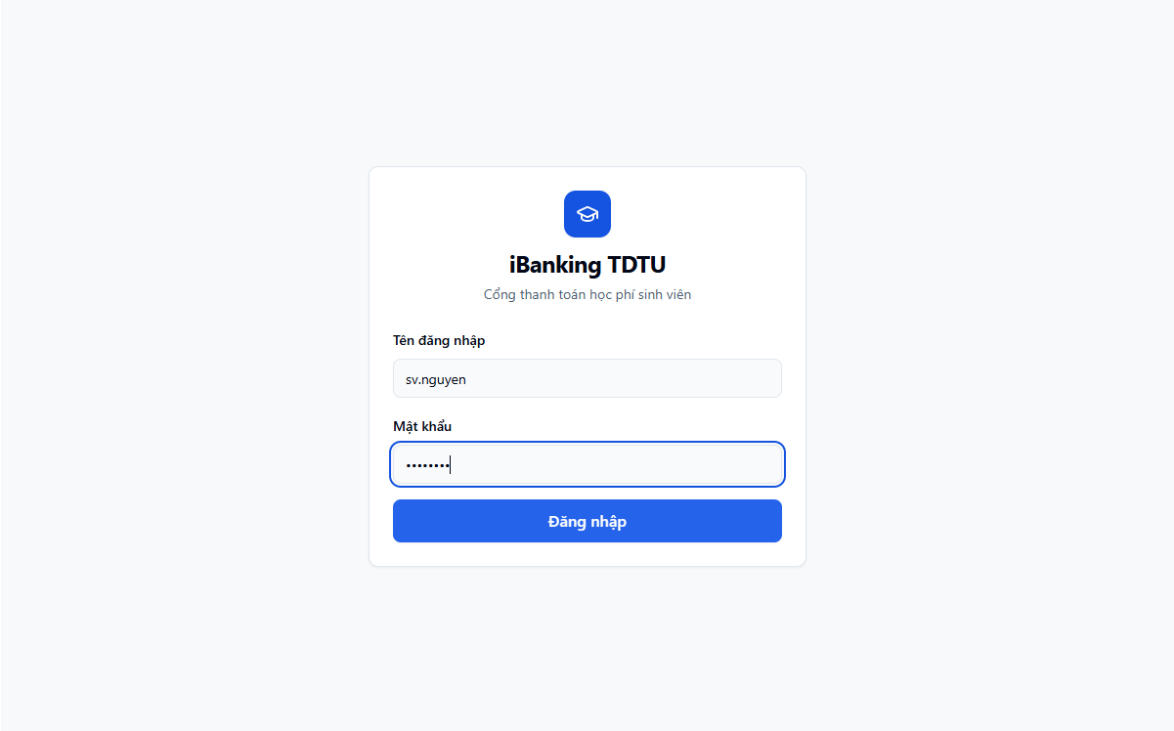
Bảng 7.1: Ma trận ánh xạ tương tác giao diện người dùng và API Microservices

| **Màn hình / Thao tác UI** | **Sự kiện kích hoạt** | **Endpoint qua Gateway (:8877)** | **Microservice xử lý** | **Mục tiêu kiến trúc & Phản hồi** |
|---|---|---|---|---|
| Giao diện Đăng nhập | Nhấn nút "Đăng nhập" | `POST /api/accounts/login` | Account Service | Xác thực mật khẩu Bcrypt, cấp phát JWT Bearer Token, lưu phiên vào AuthContext (200 OK). |
| Màn hình Thanh toán | Nhập MSSV, nhấn "Tra cứu" | `GET /api/tuitions/students/{mssv}` | Tuition Service | Truy vấn công nợ học phí, kiểm tra trạng thái UNPAID/PAID, hiển thị số tiền nộp (200 OK). |
| Màn hình Thanh toán | Tick điều khoản, nhấn "Xác nhận giao dịch" | `POST /api/payments/initiate` | Payment Service | Tạo phiên PENDING, kiểm tra số dư khả dụng, yêu cầu Notification Service gửi OTP qua Mailpit (200 OK). |
| Hộp thoại Xác thực OTP | Nhập OTP 6 số, nhấn "Xác nhận" | `POST /api/payments/confirm` | Payment Service (Orchestrator) | Xác thực OTP, gọi trừ tiền Account Service, gọi gạch nợ Tuition Service, chốt SUCCESS và gửi email biên nhận (200 OK). |
| Hộp thoại Xác thực OTP | Nhập OTP khi học phí đã có người nộp | `POST /api/payments/confirm` | Payment Service (Orchestrator) | Tuition trả về 409 Conflict, kích hoạt bù trừ hoàn tiền POST /api/accounts/refund, chốt FAILED (409 Conflict). |
| Trang Lịch sử giao dịch | Chuyển tab "Lịch sử" | `GET /api/payments/history` | Payment Service | Truy xuất toàn bộ danh sách phiên giao dịch đa trạng thái (SUCCESS / FAILED), đối soát dòng tiền minh bạch (200 OK). |

## 7.2 Giao diện đăng nhập

Giao diện đăng nhập thu thập tên đăng nhập và mật khẩu, gửi yêu cầu xác thực tới Account Service qua API Gateway và nhận token JWT khi thành công.

Biểu mẫu đăng nhập gồm hai trường bắt buộc: tên đăng nhập và mật khẩu. Hệ thống sử dụng tài khoản khởi tạo sẵn trong cơ sở dữ liệu, không có chức năng tự đăng ký. Client kiểm tra tính hợp lệ đầu vào trước khi gửi yêu cầu và hiển thị trạng thái chờ trong khi xử lý.




*Hình 7.1: Giao diện đăng nhập hệ thống iBanking*

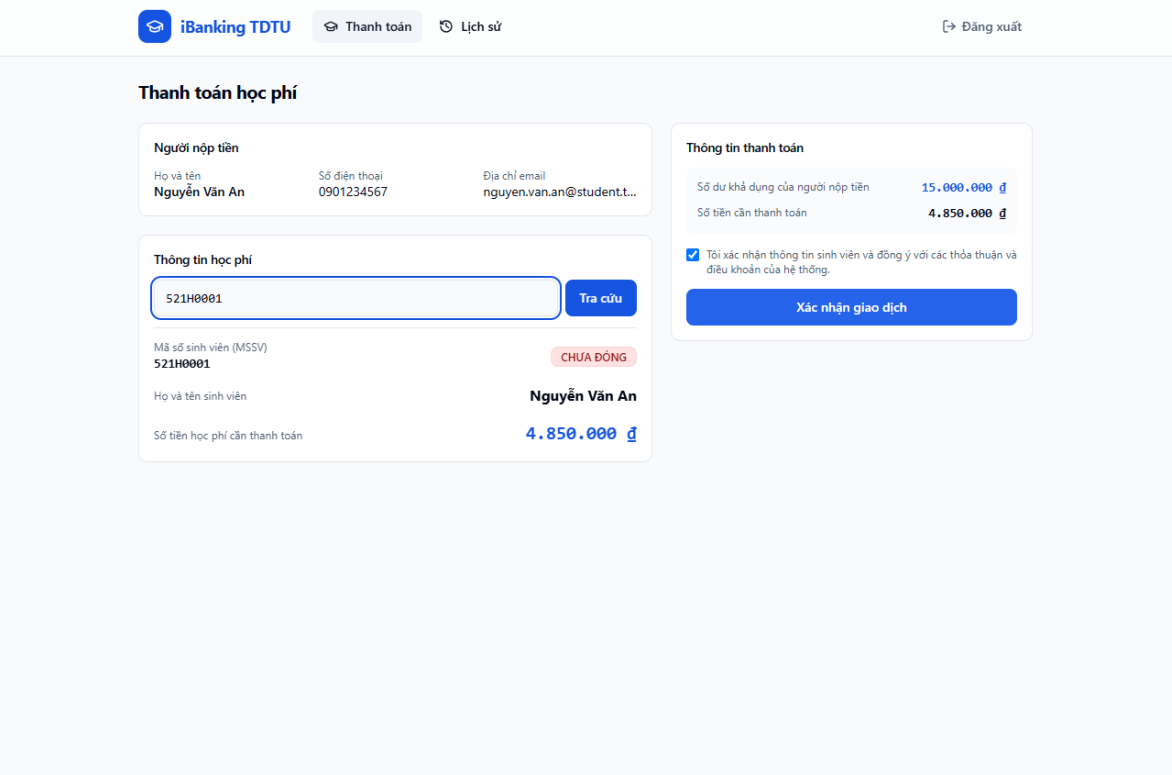
Khi xác thực thành công, mã thông báo JWT được lưu vào bộ nhớ phiên và giao diện tự động điều hướng người dùng tới phân hệ chức năng thanh toán học phí.

## 7.3 Giao diện tra cứu học phí và khởi tạo thanh toán

Màn hình thanh toán học phí là khu vực trung tâm kết hợp thông tin người nộp tiền, kết quả tra cứu công nợ sinh viên và cơ chế kiểm soát điều kiện phát sinh giao dịch tài chính theo quy định nghiệp vụ.

**Bố cục màn hình thanh toán phân chia thành ba nhóm:**

- Nhóm thông tin người nộp tiền: Tự động trích xuất từ phiên đăng nhập hiện tại gồm họ và tên, số điện thoại, địa chỉ email nhận thông báo mã xác thực và số dư khả dụng thực tế của tài khoản. Toàn bộ các trường dữ liệu định danh này ở chế độ chỉ đọc và người dùng không thể chỉnh sửa.
- Nhóm thông tin học phí sinh viên: Cung cấp ô nhập mã số sinh viên kèm nút tra cứu công nợ. Khi người dùng nhập mã số sinh viên hợp lệ, giao diện gửi yêu cầu tra cứu sang Tuition Service và lập tức hiển thị họ tên sinh viên, chuyên ngành đào tạo, học kỳ nộp và số tiền học phí bắt buộc. Hệ thống chỉ cho phép thanh toán toàn bộ khoản nợ học phí của học kỳ và không hỗ trợ thanh toán từng phần.
- **Nhóm thông tin thanh toán và điều kiện mở khóa giao dịch:** Giao diện đối chiếu số dư khả dụng với số tiền học phí cần nộp. Nút Xác nhận thanh toán chỉ được kích hoạt khi thỏa mãn đồng thời ba điều kiện khắt khe: khoản học phí tồn tại và đang ở trạng thái UNPAID, số dư khả dụng của tài khoản lớn hơn hoặc bằng số tiền học phí, và người nộp tiền đã tích chọn ô xác nhận đồng ý với điều khoản dịch vụ.

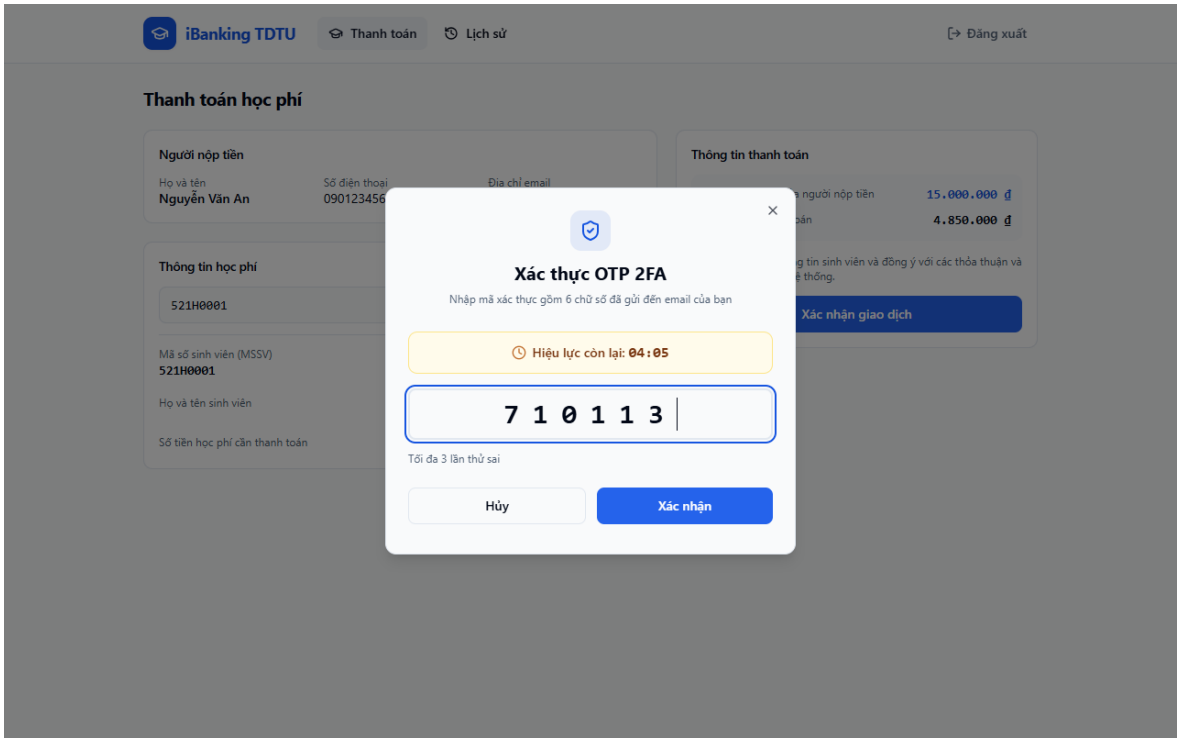



*Hình 7.2: Giao diện tra cứu công nợ học phí và thông tin thanh toán*

## 7.4 Giao diện xác thực OTP và nhận email trên Mailpit

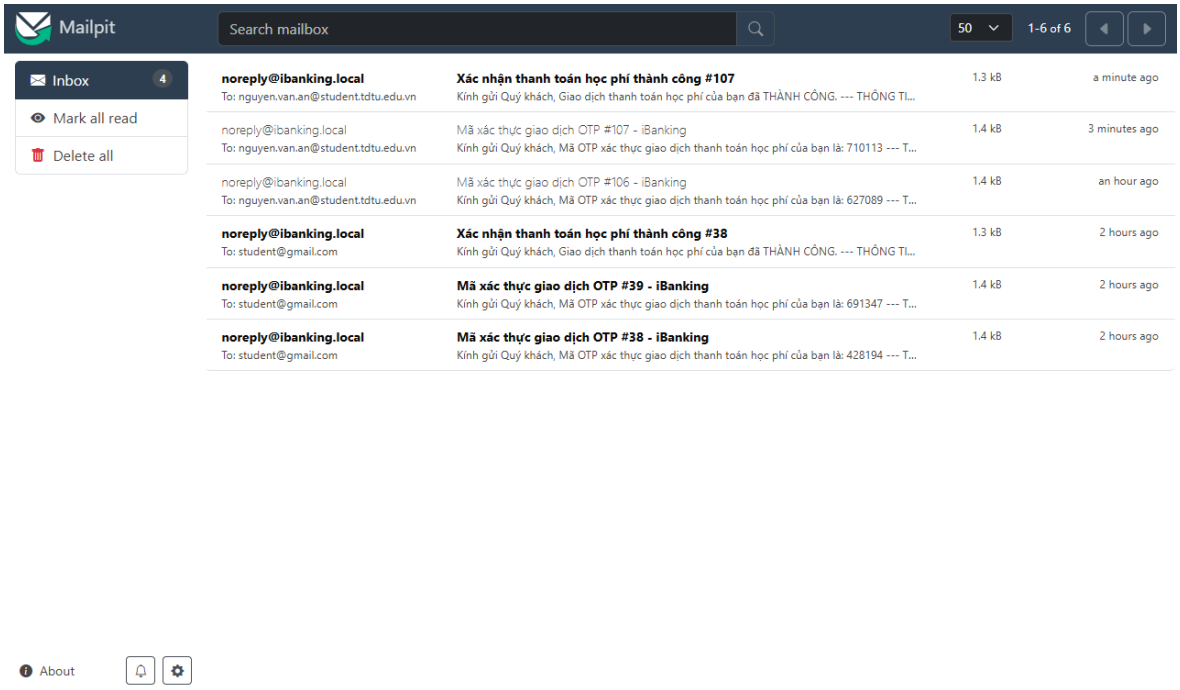
Quy trình xác thực giao dịch đa yếu tố được hiện thực thông qua hộp thoại nhập mã OTP trên ứng dụng Web kết hợp giao diện hòm thư điện tử Mailpit nhận mã xác thực theo thời gian thực.

**Đặc tả hộp thoại nhập mã xác thực và cơ chế kiểm soát an ninh:** Khi người dùng nhấn nút Xác nhận thanh toán, ứng dụng gửi yêu cầu khởi tạo phiên tới Payment Service và hiển thị hộp thoại xác thực OTP chuyên biệt. Hộp thoại cung cấp ô nhập 6 chữ số tiện lợi, hiển thị địa chỉ email người nhận và tích hợp đồng hồ đếm ngược trực quan chính xác 300 giây. Hệ thống hiển thị số lượt thử còn lại; nếu người dùng nhập sai quá 3 lần hoặc thời gian đếm ngược kết thúc, hộp thoại tự động khóa quyền thao tác và thông báo người nộp tiền thực hiện lại từ đầu.




*Hình 7.3: Giao diện hộp thoại xác thực OTP với đồng hồ đếm ngược*

**Kiểm chứng chuyển phát thư điện tử trên giao diện Mailpit:** Giao diện Webmail của máy chủ kiểm thử Mailpit tại cổng dịch vụ 8025 hiển thị trực tiếp danh sách thư điện tử do Notification Service chuyển phát qua giao thức SMTP. Tại đây, người dùng nhận được thư thông báo chứa mã xác thực OTP 6 chữ số khi bắt đầu giao dịch và thư biên nhận thanh toán ngay sau khi giao dịch hoàn tất thành công.




*Hình 7.4: Giao diện máy chủ kiểm thử Mailpit nhận thư mã OTP và biên nhận*

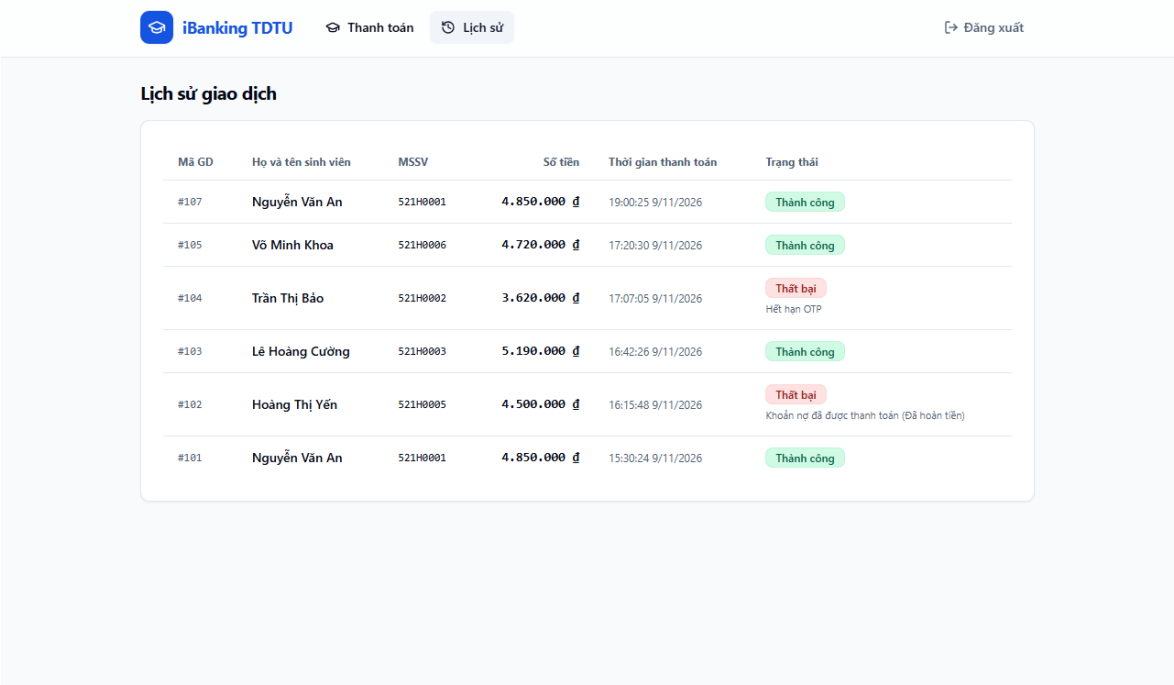
## 7.5 Giao diện kết quả biên nhận và lịch sử giao dịch

Giai đoạn hoàn tất cung cấp cho khách hàng kết quả xử lý giao dịch tức thì cùng khả năng lưu trữ, tra cứu minh bạch toàn bộ các bút toán biến động số dư và giao dịch tài chính đã thực hiện.

**Hiển thị biên lai điện tử và quản lý lịch sử thanh toán:** Ngay sau khi giao dịch gạch nợ thành công, giao diện tự động bật hộp thoại biên lai điện tử xác nhận thông tin gồm mã giao dịch thanh toán, mã số sinh viên, họ tên sinh viên, học kỳ được gạch nợ, số tiền đã khấu trừ và số dư còn lại của tài khoản. Đồng thời, toàn bộ dữ liệu này được lưu trữ vào trang Lịch sử giao dịch.

Tại trang Lịch sử giao dịch, người nộp tiền có thể tra cứu lịch sử các phiên đóng học phí đã thực hiện. Bảng lịch sử ghi nhận đầy đủ các trường dữ liệu gồm mã giao dịch, họ tên sinh viên, mã số sinh viên, số tiền thanh toán, thời gian thực thi và trạng thái nghiệp vụ. Giao diện phân định rõ ràng giữa các giao dịch 'Thành công' (nhãn xanh) và các giao dịch 'Thất bại' (nhãn đỏ) kèm nguyên nhân chi tiết như 'Khoản nợ đã được thanh toán (Đã hoàn tiền)' do kích hoạt cơ chế bù trừ hoàn tiền Saga hoặc

'Hết hạn OTP', hỗ trợ người nộp tiền và kế toán nhà trường đối soát dòng tiền một cách chính xác.




*Hình 7.5: Giao diện lịch sử giao dịch và biên nhận thanh toán học phí*

# CHƯƠNG 8. TRIỂN KHAI VÀ KIỂM THỬ HỆ THỐNG

## 8.1 Hướng dẫn cài đặt, cấu hình môi trường và khởi chạy

Hệ thống sử dụng Docker Compose và tài liệu hướng dẫn kỹ thuật chi tiết tại tệp README.md. Để phục vụ công tác nghiệm thu, đánh giá và thực nghiệm các kịch bản phân tán, dưới đây là hướng dẫn cài đặt, khởi chạy và danh mục tài nguyên truy cập trực tiếp của hệ thống:

**Yêu cầu môi trường: Hệ thống yêu cầu máy chủ cài đặt môi trường Docker Engine phiên bản 24 trở lên cùng công cụ Docker Compose phiên bản 2. Toàn bộ các thông số nhạy cảm gồm tài khoản quản trị cơ sở dữ liệu MySQL, khóa bí mật ký thẻ bài JWT_SECRET, cổng dịch vụ nội bộ và tham số máy chủ thư SMTP được thiết lập thông qua tệp biến môi trường .env tại từng thư mục dịch vụ con.**

**Khởi chạy và kiểm tra sức khỏe hệ thống: Toàn bộ cụm dịch vụ có thể khởi động bằng lệnh docker compose up --build -d. Sau khi các container hoàn tất tiến trình khởi tạo, trạng thái sẵn sàng của toàn hệ thống được kiểm chứng thông qua giao diện kiểm tra sức khỏe tại cổng API Gateway.**

```
# Câu lệnh khởi chạy toàn bộ 8 containers trong nền
docker compose up --build -d


# Kiểm tra sức khỏe toàn bộ các dịch vụ thông qua API Gateway
curl -s http://localhost:8877/health
```

```
# Phản hồi mẫu kiểm tra sức khỏe toàn hệ thống từ API Gateway
{
  "status": "UP",
  "gateway": "healthy",
  "services": {
    "account_service": "UP",
    "tuition_service": "UP",
    "payment_service": "UP",
    "notification_service": "UP"
  }
}
```

**Quy trình thực nghiệm và kiểm tra sản phẩm bao gồm các bước sau:**

• Bước 1 - Yêu cầu môi trường tiên quyết: Máy chủ cần cài đặt sẵn Docker Engine 24+ và Docker Compose v2 (hoặc Docker Desktop). Đối với môi trường kiểm thử tự động bằng Python, yêu cầu môi trường Python 3.11+ kèm thư viện httpx.

• Bước 2 - Khởi chạy toàn bộ cụm dịch vụ: Tại thư mục gốc dự án d:\project\soa, thực thi lệnh `docker compose up --build -d`. Lệnh này sẽ tự động tải các image nền, biên dịch mã nguồn của 4 microservices backend, cổng API Gateway và ứng dụng Frontend React SPA, đồng thời khởi tạo cơ sở dữ liệu MySQL và máy chủ Mailpit.

• Bước 3 - Truy cập và nghiệm thu trực quan trên trình duyệt:

| Thành phần | Địa chỉ truy cập (URL) | Thông tin đăng nhập / Mục đích |
|---|---|---|
| Web Frontend | `http://localhost:3000` | Đăng nhập tài khoản demo: sv.nguyen / Test@123 (Số dư ban đầu: 7.500.000 đ) |
| Mailpit Web UI | `http://localhost:8025` | Hòm thư nhận mã OTP 2FA và nhận email biên nhận sau thanh toán |
| API Gateway Health | `http://localhost:8877/health` | Kiểm tra sức khỏe toàn diện và tính sẵn sàng của 4 microservices |
| Cơ sở dữ liệu MySQL | `localhost:8552 (root / rootpassword)` | Truy xuất 4 cơ sở dữ liệu phân tán: account_db, tuition_db, payment_db, notification_db |

• Bước 4 - Chạy bộ kiểm thử tự động kịch bản phân tán: Mở một cửa sổ dòng lệnh tại thư mục gốc và thực thi lệnh `python backend/tests/test_concurrency.py`. Bộ kiểm thử sẽ tự động thực hiện trọn vẹn 4 kịch bản kiểm tra an ninh luồng điều phối, chống thấu chi tài khoản, bù trừ hoàn tiền Saga và khóa chống gửi trùng Idempotency-Key.

## 8.2 Đóng gói và tích hợp hệ thống bằng Docker Compose

Tám thành phần của hệ thống bao gồm bốn microservices nghiệp vụ, cổng API Gateway, ứng dụng giao diện Web, hệ quản trị cơ sở dữ liệu MySQL và máy chủ thư điện tử Mailpit được đóng gói thành các container độc lập và quản lý tập trung thông qua tệp docker-compose.yml.

Bảng 8.1: Cấu hình triển khai 8 containers trong Docker Compose

| **Tên dịch vụ container** | **Công nghệ nền tảng** | **Cổng ánh xạ máy chủ và container** | **Điều kiện phụ thuộc khởi động** | **Vai trò trong hệ thống** |
|---|---|---|---|---|
| mysql | mysql:8.0 | 8552:8552 | - | Hệ quản trị cơ sở dữ liệu lưu trữ bốn lược đồ độc lập trên vùng lưu trữ |
| mailpit | axllent/mailpit | 1025:1025, 8025:8025 | - | Máy chủ SMTP nhận thư điện tử mã OTP cùng biên nhận và giao diện Webmail |

| Tên dịch vụ container | Công nghệ nền tảng | Cổng ánh xạ máy chủ và container | Điều kiện phụ thuộc khởi động | Vai trò trong hệ thống |
|---|---|---|---|---|
| account-service | Python 3.11 với FastAPI | 127.0.0.1:8661:8661 | mysql đã sẵn sàng | Quản lý tài khoản, xác thực JWT, trừ tiền, hoàn tiền và sổ cái kế toán |
| tuition-service | Python 3.11 với FastAPI | 127.0.0.1:8732:8732 | mysql đã sẵn sàng | Tra cứu công nợ học phí theo mã số sinh viên và gạch nợ nguyên tử |
| notification-service | Python 3.11 với FastAPI | 127.0.0.1:8940:8940 | mysql và mailpit | Sinh mã OTP 6 số, kiểm soát hạn 5 phút và gửi email qua SMTP |
| payment-service | Python 3.11 với FastAPI | 127.0.0.1:8815:8815 | các dịch vụ backend phụ thuộc | Bộ điều phối giao dịch Saga Orchestrator và lưu lịch sử thanh toán |
| api-gateway | Python 3.11 với FastAPI | 8877:8877 | các dịch vụ backend | Cổng Reverse Proxy, kiểm tra JWT, Workflow Guard và Idempotency |

| Tên dịch vụ container | Công nghệ nền tảng | Cổng ánh xạ máy chủ và container | Điều kiện phụ thuộc khởi động | Vai trò trong hệ thống |
|---|---|---|---|---|
| frontend | Node 20 và máy chủ Nginx | 3000:3000 | api-gateway | Giao diện người dùng Web React phục vụ qua máy chủ Nginx |

**Cơ chế cô lập mạng và thứ tự khởi động phụ thuộc: Hệ thống thiết lập mạng ảo riêng biệt soa-net kết nối tất cả các container nội bộ. Các cổng nội bộ của bốn microservices nghiệp vụ được cấu hình ràng buộc vào địa chỉ loopback cục bộ nhằm ngăn chặn hành vi truy cập trực tiếp từ mạng bên ngoài mà bắt buộc phải đi qua cổng API Gateway tại cổng 8877. Thứ tự khởi động được kiểm soát thông qua chỉ thị depends_on kết hợp bài kiểm tra sức khỏe của container MySQL, bảo đảm các cơ sở dữ liệu đã nạp xong kịch bản init.sql thì các tiến trình dịch vụ backend mới bắt đầu tiếp nhận kết nối.**

## 8.3 Kiểm thử tự động các kịch bản giao dịch đồng thời

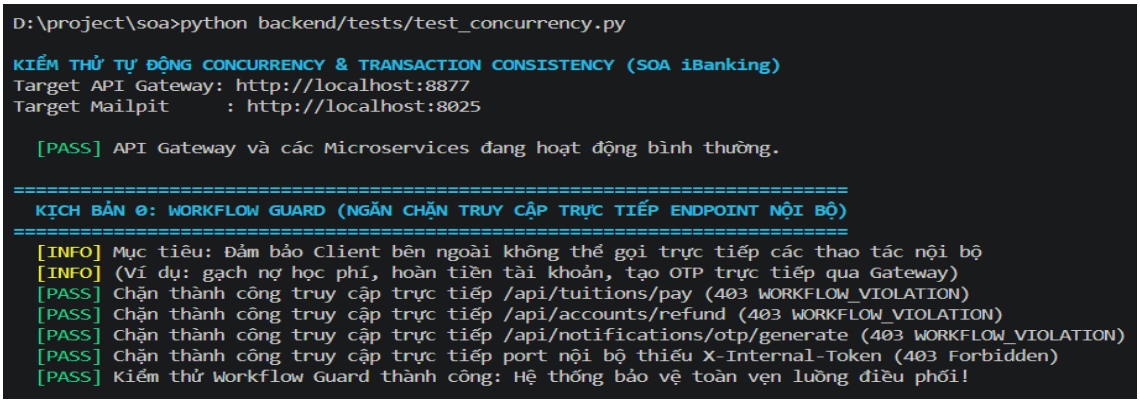
Bộ kiểm thử tự động trong tệp test_concurrency.py sử dụng thư viện HTTP client bất đồng bộ httpx nhằm kiểm chứng cơ chế khóa bi quan và quy trình bù trừ Saga.

```
# Câu lệnh thực thi bộ kiểm thử tự động các kịch bản đồng thời
pip install httpx
python backend/tests/test_concurrency.py
```

### *8.3.1 Kịch bản 0: Bảo vệ luồng nghiệp vụ Workflow Guard*

Kiểm chứng rằng người dùng bên ngoài, kể cả khi sở hữu thẻ bài JWT hợp lệ, không thể gọi trực tiếp vào các endpoint nội bộ gồm gạch nợ học phí, hoàn tiền tài khoản hoặc sinh mã OTP.

**Khi gửi yêu cầu trực tiếp vào các đường dẫn nội bộ thông qua API Gateway, hệ thống chặn tại tầng bộ lọc và phản hồi mã trạng thái 403 Forbidden kèm mã lỗi WORKFLOW_VIOLATION. Toàn bộ các thử nghiệm đều vượt qua thành công như minh họa trên Hình 8.1.**

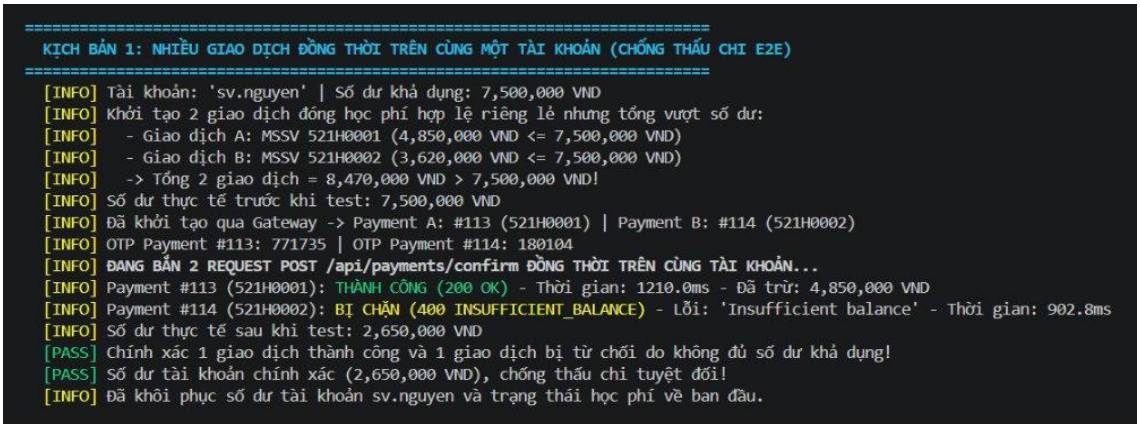



*Hình 8.1: Kịch bản 0 - Chặn truy cập trực tiếp API nội bộ tại API Gateway*

### *8.3.2 Kịch bản 1: Giao dịch đồng thời trên cùng tài khoản*

Kiểm chứng tình huống tranh chấp đồng thời khi một tài khoản phát sinh nhiều giao dịch thanh toán có tổng giá trị vượt quá số dư khả dụng, hệ thống phải ngăn chặn việc chi tiêu vượt hạn mức.

**Tài khoản người dùng có số dư ban đầu 7.500.000 VND phát sinh đồng thời hai giao dịch đóng học phí trị giá 4.850.000 VND và 3.620.000 VND với tổng số tiền 8.470.000 VND vượt quá số dư hiện có. Cả hai phiên đều nhận mã OTP riêng biệt và gửi yêu cầu xác nhận cùng một thời điểm. Nhờ cơ chế khóa bi quan mức dòng tại Account Service, đúng một giao dịch được phê duyệt thành công với mã 200 OK làm số dư giảm xuống 2.650.000 VND, giao dịch còn lại bị từ chối với mã lỗi 400 INSUFFICIENT_BALANCE. Kết quả thực thi thành công được thể hiện trên Hình 8.2.**

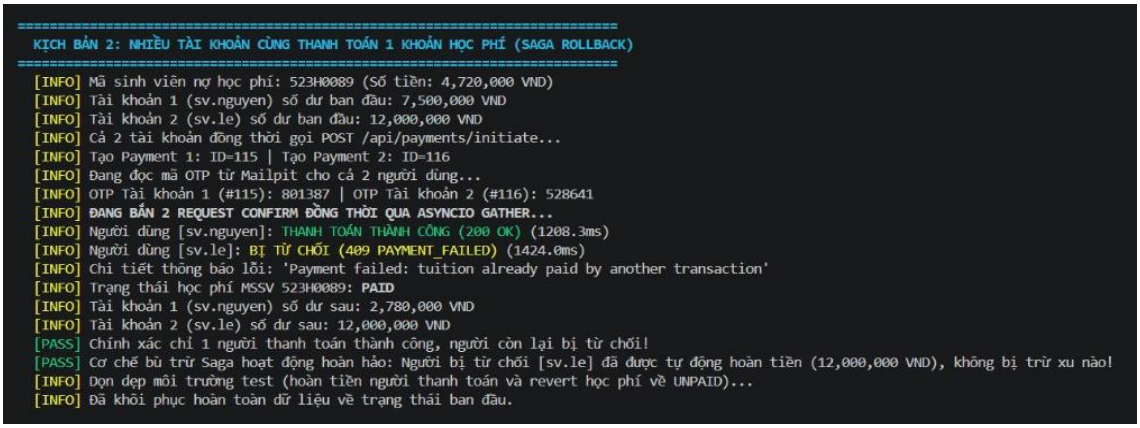



*Hình 8.2: Kịch bản 1 - Giao dịch đồng thời trên cùng một tài khoản*

### *8.3.3 Kịch bản 2: Đồng thời đóng học phí và bù trừ Saga*

Kiểm chứng tình huống tranh chấp đồng thời khi hai tài khoản khác nhau cùng xác nhận thanh toán cho một khoản học phí sinh viên tại cùng một thời điểm, bảo đảm khoản học phí chỉ được gạch nợ đúng một lần và tài khoản nộp chậm được hoàn tiền tự động.

**Hai tài khoản người nộp khác nhau cùng gửi yêu cầu xác nhận thanh toán cho sinh viên mã số 523H0089. Giao dịch chiếm khóa trước gạch nợ thành công với mã 200 OK và chuyển trạng thái học phí sang PAID. Giao dịch đến sau bị xung đột tại Tuition Service và nhận mã lỗi 409 Conflict. Khối điều phối Payment Service lập tức kích hoạt nhánh bù trừ Saga, hoàn trả lại toàn bộ số tiền đã khấu trừ về tài khoản nguồn và ghi nhận bút toán REFUND. Khoản nợ học phí chỉ bị thanh toán đúng một lần duy nhất như minh họa trên Hình 8.3.**

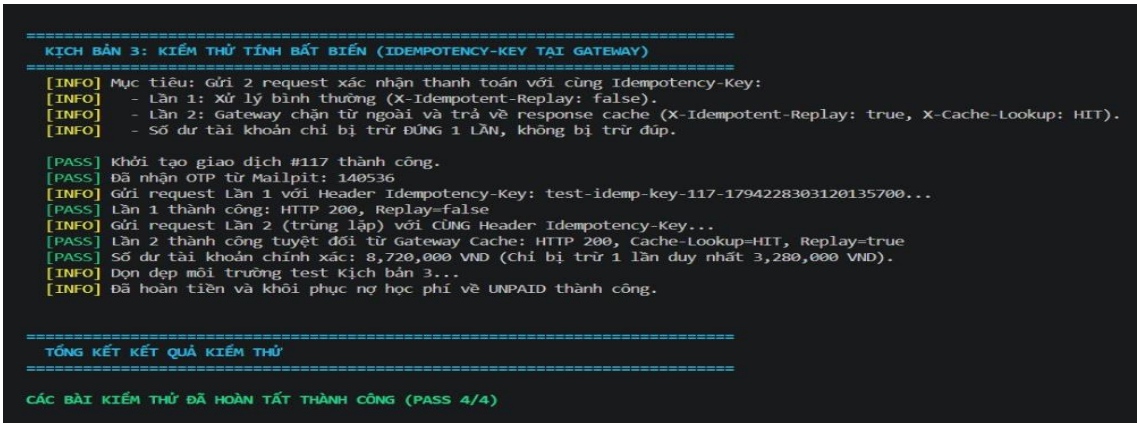



*Hình 8.3: Kịch bản 2 - Hai tài khoản cùng đóng một khoản học phí và bù trừ Saga*

### *8.3.4 Kịch bản 3: Chống gửi trùng yêu cầu Idempotency-Key*

**Kiểm chứng rằng khi người dùng nhấp đúp chuột hoặc đường truyền mạng phát sinh hai yêu cầu xác nhận thanh toán trùng lặp mang cùng khóa Idempotency-Key, hệ thống chỉ xử lý giao dịch đúng một lần.**

**Yêu cầu đầu tiên được chuyển tiếp xuống backend xử lý bình thường. Yêu cầu trùng lặp thứ hai được API Gateway đối soát khóa trong bộ nhớ đệm và hoàn trả ngay kết quả lưu sẵn Cache Replay mà không phát sinh thêm bất kỳ giao dịch trừ tiền nào. Kết quả thực thi thành công được minh chứng trên Hình 8.4.**




*Hình 8.4: Kịch bản 3 - Khóa bất biến Idempotency-Key tại API Gateway*

# TÀI LIỆU THAM KHẢO

Tiếng Việt

[1] Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng, 2026. Tài liệu bài giảng và Đặc tả môn học Kiến trúc hướng dịch vụ 504070.

Tiếng Anh

[2] Newman, S., 2021. Building Microservices: Designing Fine-Grained Systems, tái bản lần 2. O'Reilly Media.

[3] Richardson, C., 2018. Microservices Patterns: With Examples in Java, Chương 4: Quản lý giao dịch phân tán với Saga. Manning Publications.

[4] FastAPI Documentation, 2026. FastAPI Framework - High performance, easy to learn, fast to code, ready for production. Truy cập từ https://fastapi.tiangolo.com/

[5] Oracle Corporation, 2026. MySQL 8.0 Reference Manual - InnoDB Locking and Transaction Model. Truy cập từ https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-transaction-model.html